SECP2523: DATABASE

SEM I 2024/2025

DATABASE DESIGN DESCRIPTIONS: LOGICAL DATABASE DESIGN

FOR KADA Online System

Group Name: Technicrab

Members:

1. Cheryl Cheong Kah Voon A23CS0060
2. Chau Ying Jia A23CS0213
3. Neo Li Xin A23CS0253
4. Lau Yee Wen A23CS0099
5. Woo Cheng Shuan A23CS0283

Stakeholder: Lembaga Kemajuan Pertanian Kemubu (KADA)

Printing Date: 27 January 2025

TABLE OF CONTENTS

NOTE: A module can consist of one or multiple (related) use case. A team member in a group must be responsible to at least one module.

1. Introduction

1.1 Overview about the company

Koperasi Kakitangan KADA Kelantan Berhad, a subsidiary of Lembaga Kemajuan Pertanian Kemubu (KADA) under the Ministry of Agriculture and Food Security, aims to improve the economic and social well-being of farmers in the Kemubu region. The cooperative provides its members with essential financial and administrative services, as well as the tools and resources they need to grow sustainably.

To streamline operations and improve efficiency, the cooperative intends to move away from its current manual processes and towards a comprehensive digital platform tailored to its organisational goals.

1.2 The existing system

The existing system operates manually, which limits its efficiency and productivity. Nevertheless, it does offer several main services and features, as listed below:

1. **Membership Registration**

   This is a structured process for KADA's staffs to become part of their members, granting them accessible to a variety of cooperative benefits and services.

   The current member registration system relies on a manual process, necessitating individuals to complete information on printed forms. During registration, various personal details, such as name, identification number, address, and others are gathered. Afterward, KADA staff will need to review the registration documents again. This method is not only time-consuming but also susceptible to errors and places a significant strain on human resources.

Figure 3.1 printed Member Registration Form

2. Loan Applications

Their loan application seeks financial assistance for personal, educational, or business needs. The goal of the loan application process is to be clear and efficient, allowing cooperative members to obtain the required funds quickly. Currently, the loan application system is manual too, necessitating that information be entered on printed forms. In addition, the qualified members are not able to check on their past transactions and records that is hard for them to look back for their total loans and payments if they lost their statements.

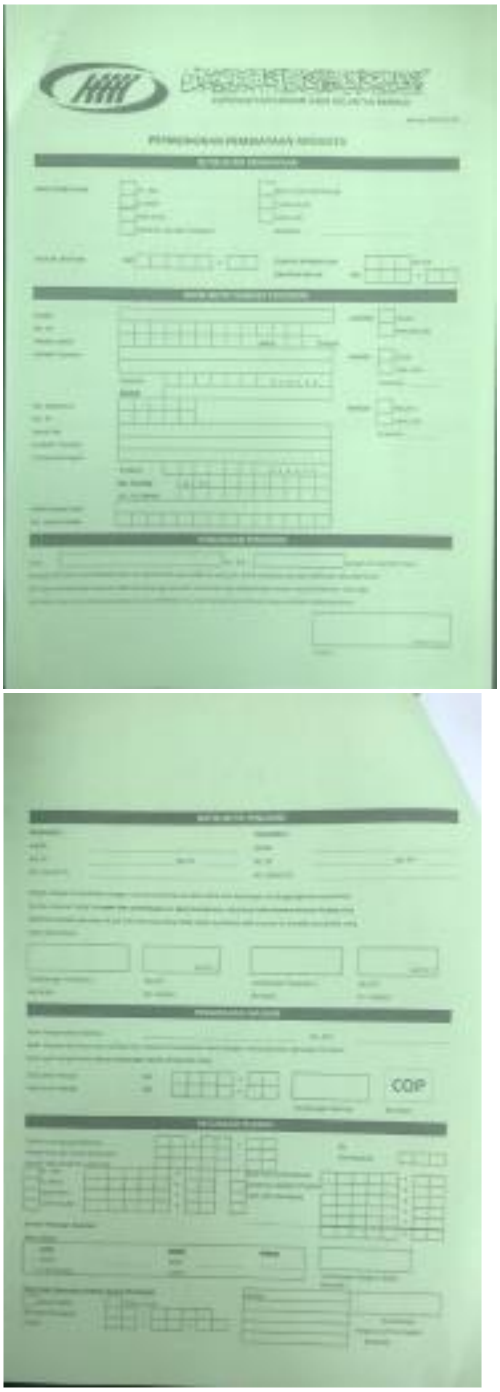


Figure 3.2 Member Loan Applications Form

3. Member Financial Statements

Koperasi Kakitangan KADA Kelantan Berhad currently uses a manual system for generating and distributing member financial statements. This involves collecting financial data manually from various records and compiling it into individual member statements. These statements include details such as the member's name, IC number, shares held, loans taken, and savings.

Once the statements are prepared, they are reviewed by staff to ensure accuracy. After verification, the statements are handed out to members. Members then sign and return the verification portion to confirm receipt and accuracy.

This manual process is time-consuming, labor-intensive, and prone to errors due to manual data entry. It also places a significant burden on the cooperative's limited human resources and delays members' access to their financial information. Implementing an automated system would streamline this process, reduce staff workload, and provide members with timely and accurate financial information.

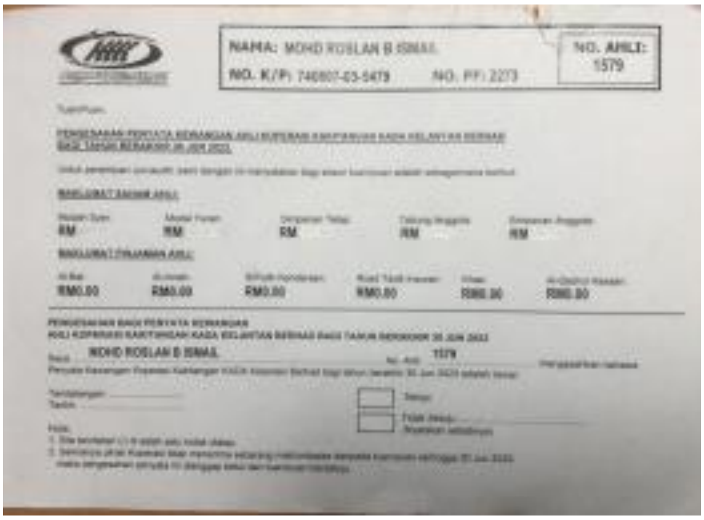

NAMA: MOHD ROSLAN B ISMAIL
NO. K/P: 740807-03-5479 NO. PF: 2273
NO. AHLI: 1579

Figure 3.3 Member Financial Statement

Problem with Existing System

The registration system at KADA company is currently handled manually, showing signs of low efficiency and productivity, as described below. Even though it offers only a limited number of essential services and features, there is room for improvement. Recognizing the constraints of the existing system emphasizes the necessity for a complete overhaul to enhance the system. Moving towards a new computerized system could greatly elevate their services and features. This new system will bring about automation, streamlined processes, an improved website for better performance, and an expanded array of services for users.

1. Insufficient registration process

   In the current system, all registration procedures are manually processed, like customers filling out paper forms to register and staff handling each form individually. Personal information, both private and sensitive, is predominantly stored in hard copies during registration. This method not only poses security risks but also makes it challenging to retrieve or locate information when necessary. Additionally, the registration papers require further collection, rearrangement and scrutiny by staff at the end. It really takes much more time than doing the registration online.

2. Inconvenient Status Checking

In the existing process, members are unable to check the status directly, they must either call or wait for an email from the company to obtain the current status, whether it is about the registration approval or payment status. This method is not that convenient, as staff have to manually send emails instead of having an automated system on their website that segregates the status. Improvements are necessary to modernize this outdated system to enhance the efficiency of the processes.

3. Heavy Dependence on Human Resources

The existing manual system faces an issue as it heavily depends on human resources. All the steps starting from the registration and application for the loan are done manually. If the number of human resources decreases beyond a certain point, it will impact productivity and efficiency in the workplace. To address the issue of heavy dependence on human resources, the implementation of automation solutions is crucial as it can help streamline processes, minimise errors, and free up employees to focus on more strategic tasks that require human intervention.

4. Inefficient and human error

We know that retrieving and managing physical documents is time-consuming and the manual system heavily depends on human resources. This might lead to the chances of human error rising when staff forget to email the current status, missing payment receipts, or failing to gather customer information and also missing information. These inefficiencies will cause delays in service delivery and mistakes in data processing, affecting overall customer satisfaction and also might disrupt the entire development process as incomplete or inaccurate data may result. Therefore, the company requires a more well-designed system.

5. Security Concerns of Manually Stored Customer Forms and Information

In the digital age, safeguarding customer information is paramount for maintaining trust and ensuring compliance with data protection regulations. The cooperative currently relies on manual storage of customer forms and information in physical cupboards. This method poses significant security risks that need urgent attention to protect sensitive data and uphold our commitment to customer privacy. As we can see, employees, cleaning staff, and visitors may easily access the cupboards, potentially leading to data breaches. Furthermore, it is challenging to track who accessed or modified the physical documents, making it difficult to audit and ensure the accountability of data handling practices.

Features	**Existing System**	**Proposed System**
Membership registration	Yes	Yes
Loan application	Yes	Yes

Application status checking	No	Yes
Check financial status and statements	Yes	Yes
Check monthly and annually report	No	Yes
Check profit and loss from account	No	Yes

1.3 The proposed system

The proposed Koperasi KADA Online System is a fully integrated digital platform aimed at enhancing the efficiency, security, and overall user experience of Koperasi Kakitangan KADA Kelantan Berhad. It replaces the current manual processes, which are time-consuming, prone to errors, and heavily reliant on human resources.

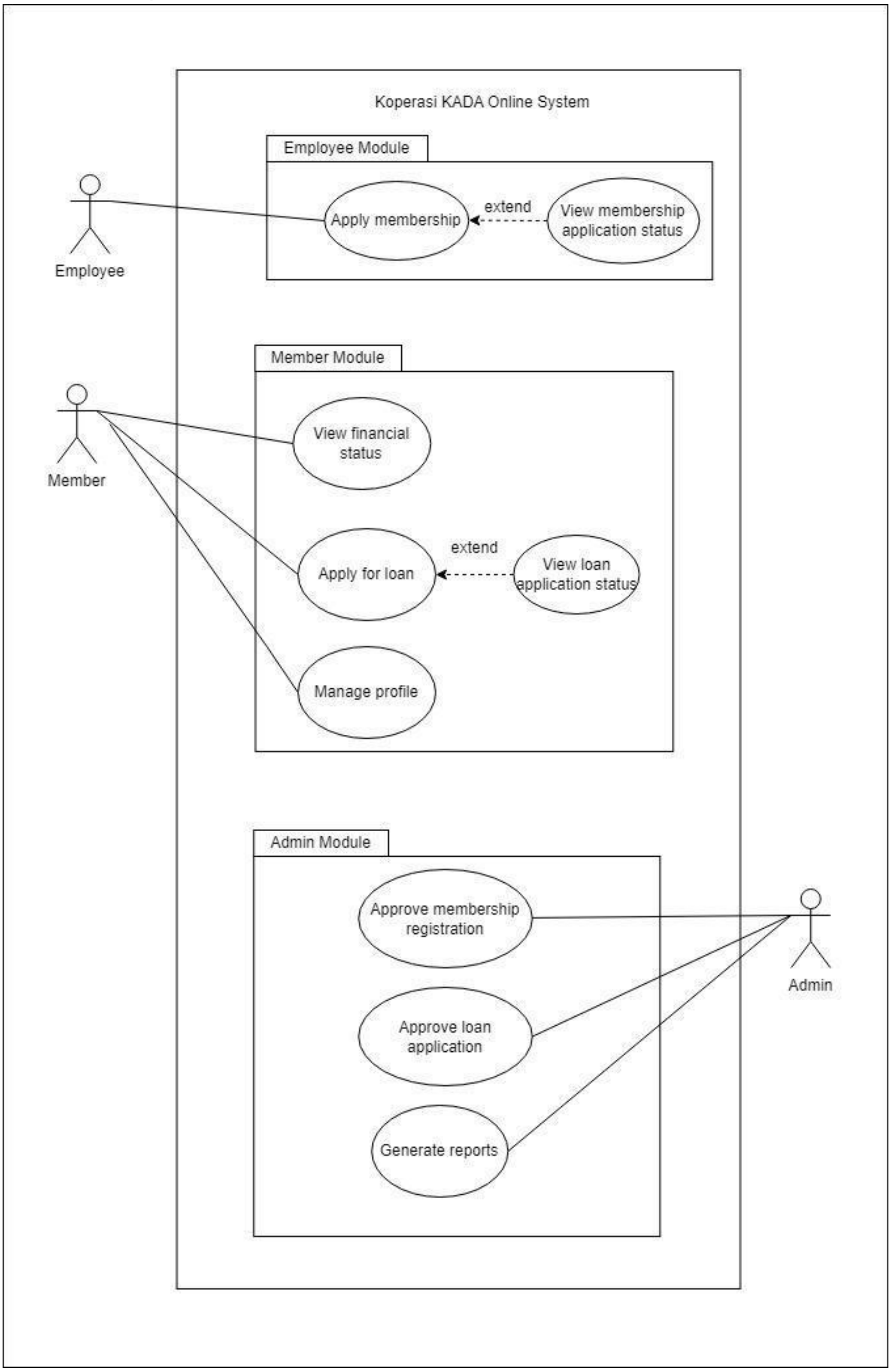
The system allows new users to register online by submitting personal details such as name, address, and identification data. Upon submission, this data is processed automatically, verified for accuracy, and stored securely in a digital database, and provides real-time feedback on the status of the application, whether it is successful, pending, or rejected. Once a user becomes a registered member, they gain access to several convenient services, such as checking the real-time status of their registration, loan applications, and payments through an online portal. Members can also manage their savings, view financial statements, and track potential dividends from their investments, receiving automated email or SMS notifications regarding their financial activities.

For the administrative staff (KADA employees), the system reduces the burden of manual data handling and processing. Admin are responsible for reviewing member applications, verifying submitted information, and authorizing loan applications based on pre-set criteria. For the admin, the system simplifies tasks by providing a centralized platform for reviewing member applications and loan requests. Admins can verify application details, approve or reject submissions based on eligibility criteria, and manage the digital records securely, reducing the need for paper-based processes. They can also update the status of applications and ensure that members are promptly notified of any changes. The system minimizes human error by automating many of the routine processes, freeing up staff to focus on more strategic and high-priority tasks.

Board Directors, who oversee high-level decisions, are involved in reviewing and approving larger or more critical loan applications. Their role is to ensure that these applications align with the cooperative's strategic goals and regulatory requirements. They have the authority to make final decisions on both member and loan applications, with their approval or rejection logged into the system for transparency and future reference.

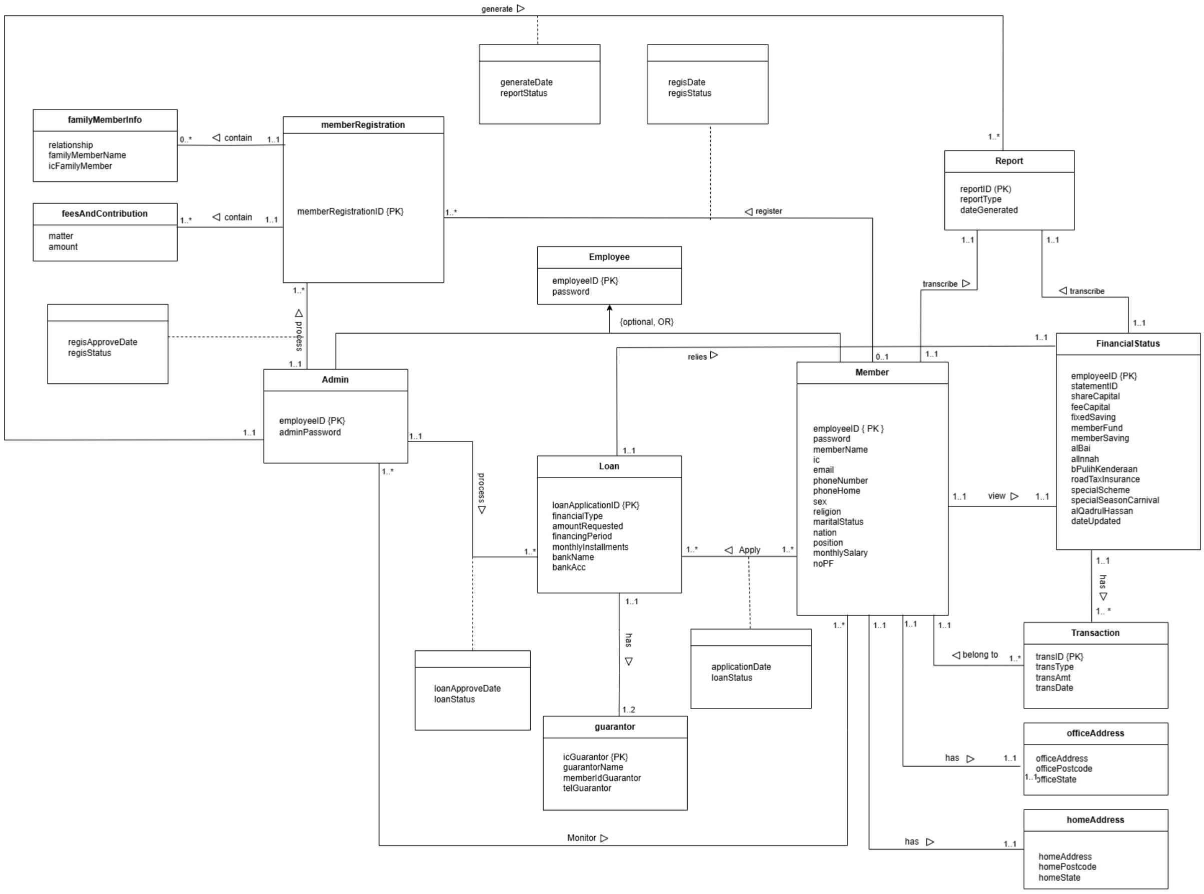
Overall, the KADA Registration System brings about a more efficient, secure, and user-friendly environment for all stakeholders. It reduces the burden on human resources, improves data management, enhances communication, and ensures that sensitive member information is stored securely. Also, it enhances security measures through features like Two-Factor Authentication (2FA), and providing a seamless experience for users. By automating key functions and providing a streamlined user interface, the system aims to elevate the cooperative's service delivery and operational capabilities, fostering greater member satisfaction and organizational productivity.

Use Case Diagram for Proposed System



2. Global Conceptual Data Model (Global ERD)

2.1 Global Conceptual Database Design (Conceptual ERD)



2.2 Data Dictionary for the global conceptual ERD.

2.2.1 Identifying Entity

Entity Name	Description	Aliases	Occurrence
employee	Represents any person accessing the KADA system	New user, guest	Occurs when first-time user visit the website
member	Represents registered users who have created KADA accounts	Members	Occurs when new user or guest register as a member
admin	Represents the system admin who responsible for managing KADA system	Administrator, staff, board director	Occurs when system built and processing members' account activity
memberRegistration	Describe all activities when a user register as a member	Users	Occurs when a new user register as a member through KADA system
familyMemberInfo	Represents all the information about members' family members recorded	Members' family record	Occurs when user register as a member
feesAndContribution	Represents fees paid by members and any contributions made towards KADA	Contribution and fees record	Occurs when user register as a member
loan	Represents loan applications submitted by members	Loan application record	Occurs when member apply for loan
guarantorDetails	Represents information about members' loan guarantors	Members' loan application guarantor details	Occurs when member apply for loan
financialStatus	Represents member's financial health	Members' financial reports / records	Occurs when member make a loan application and to check financial status
transaction	Represents all financial transactions made by members	Accounts loan / fees transaction	Occurs when admin make an report to be viewed by members
report	Stores generated reports related to financial summaries, transaction	Monthly / yearly reports, financial reports	Occurs when a monthly or yearly report have to

	histories, or member activities		be published to the members by admin
officeAddress	Stores members' office address	Members' personal information	Occurs when employer register as a member
homeAddress	Stores members' house address	Members' personal information	Occurs when employer register as a member

2.2.2 Identifying Relationship Types

Entity Name	**Multiplicity**	**Relationship**	**Entity Name**	**Multiplicity**
member	0..1 1..1 1..1 1..1 1..*	Registers Has Has view Applies	memberRegistration homeAddress officeAddress financialStatus loan	1..1 1..1 1..1 1..1 1..*
admin	1..1 1..* 1..* 1..1	approves generate Monitor process	memberRegistration report member loan	1..* 1..1 1..* 1..*
memberRegistration	1..1 1..1	has has	familyMemberInfo feesAndContribution	0..* 1..*
loan	1..1 1..1	Has relies	guarantorDetails financialDetails	1..2 1..1
financialStatus	1..1	has	transaction	1..*
transaction	1..*	belong to	member	1..1

2.2.3 Attributes Description

Entity Name	**Attributes**	**Data Type & Length**	**Description**	**Nulls**	**Multi-valued**	**Example**
employee	employeeID(PK)	INT(4)	Unique ID for the employee	No	No	0001
	password	VARCHAR (30)	User's account password	No	No	P12345W
member	employeeID (PK)	INT(4)	Unique ID for member	No	No	0001
	password	VARCHAR (30)	Password for member login	No	No	P12345W
	memberName	VARCHAR (80)	Name of the member	No	No	Ali Abu
	ic	CHAR (12)	Identification number of the member	No	No	040101081234
	email	VARCHAR (30)	Member's email address	No	No	member@gmail.com
	phoneNumber	VARCHAR (15)	Contact number of the member	No	No	+60123456789
	sex	CHAR (1)	Gender of the member (M/F)	No	No	Female
	religion	VARCHAR (30)	Religion of the member	Yes	No	Buddha
	maritalStatus	VARCHAR (15)	Marital status	Yes	No	Single
	nation	VARCHAR (30)	Nationality of the member	Yes	No	Malaysia
	position	VARCHAR (30)	Working position of the member	No	No	Data Engineer
	monthlySalary	DECIMAL (10,2)	Member's monthly salary	No	No	10000.00

	noPF	INT(4)	Member no PF	No	No	5678
admin	employeeID (PK)	INT(4)	Unique ID for admin	No	No	4567
	adminPassword	VARCHAR (20)	Password of the admin	No	No	noahlim
memberRegistration	memberRegistrationID (PK)	VARCHAR (20)	Unique ID for member registration	No	No	MR123456789
familyMemberInfo	relationship	VARCHAR (30)	Relationship between family member and member	Yes	No	Daughter
	familyMemberName	VARCHAR (80)	Name of the family member	No	No	HenryWong
	icFamilyMember	CHAR (12)	Identification number of the family member	No	No	040101084567
feesAndContribution	entryFee	INT(4)	Entry fee	No	No	200.00
	modalShare	INT(11)	Modal share for member	No	No	200.00
	feeCapital	INT(11)	Member's fee capital	No	No	200.00
	deposit	INT(11)	Member's deposit	No	No	200.00
	contribution	INT(11)	Member's contribution	No	No	200.00
	fixedDeposit	INT(11)	Member's fixed deposit	No	No	200.00
	others	INT(11)	Others contribution	No	No	200.00

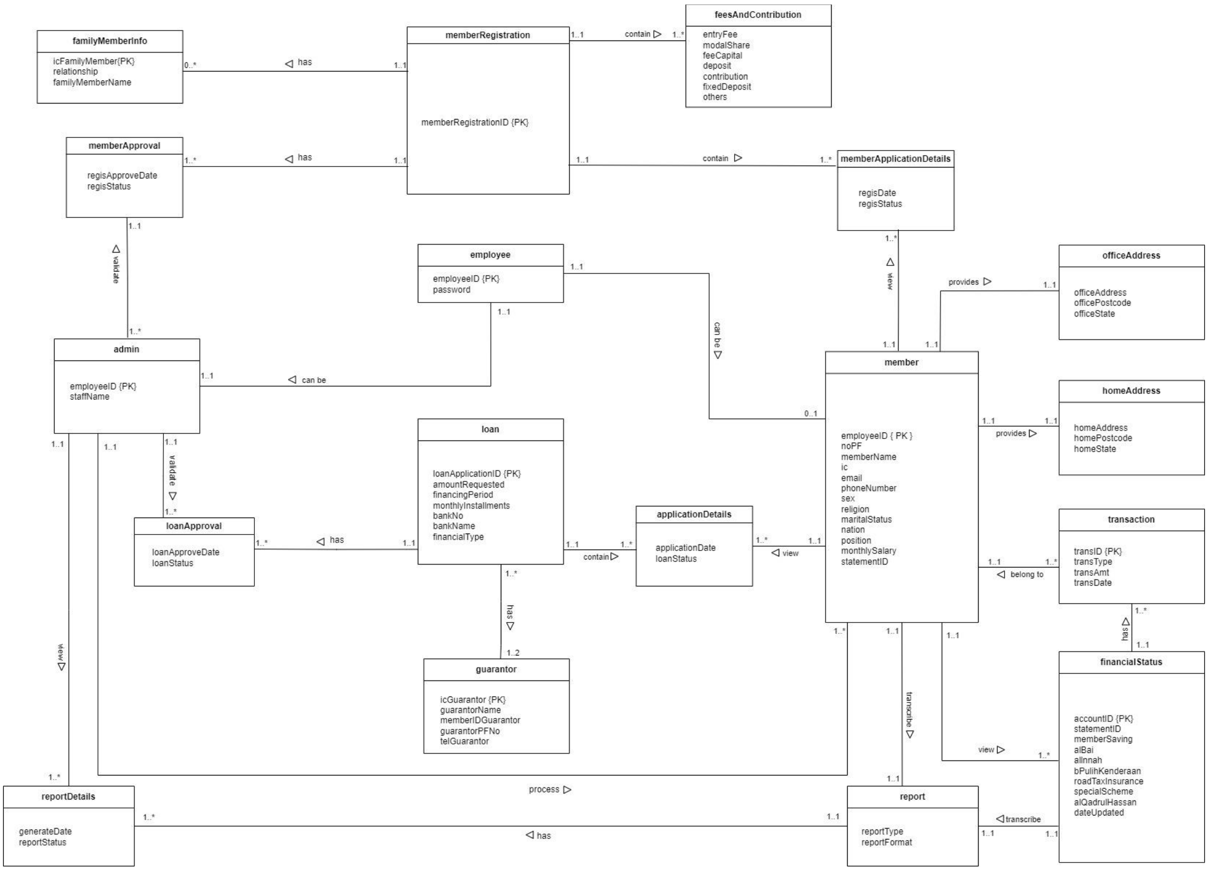
loan	loanType	VARCHAR(30)	Type of loan	No	No	AL_BAI
	loanApplicationID (PK)	VARCHAR (20)	Unique ID for loan application	No	AUTO-INCREMENT	7777
	amountRequested	INT (12,2)	Amount requested for the loan	No	No	30000.00
	financingPeriod	INT(11)	Loan repayment period in months	No	No	24
	monthlyInstallments	DECIMAL (10,2)	Monthly repayment amount	No	No	650.00
	bankName	VARCHAR (50)	Bank name for loan transfer	No	No	RHB
	bankAccount	VARCHAR (20)	Bank account number	No	No	01234567890
guarantor Details	guarantorName (PK)	VARCHAR (80)	Name of the guarantor	No	No	Evelyn Lee
	icGuarantor	CHAR (12)	Identification number of the member	No	No	040101081234
	memberIDGuarantor	VARCHAR (8)	Unique ID for member's guarantor	No	No	8888
	telGuarantor	VARCHAR (15)	Contact number of the member' guarantor	No	No	0123456789
financialStatus	employeeID (PK)	INT(4)	Unique ID for employee	No	No	5522
	statementID	INT (20)	Unique ID for financial statement	No	No	12345678901

shareCapital	DECIMAL (12,2)	Total share capital for the member	No	No	2000.00
feeCapital	DECIMAL (10,2)	Total fee capital for the member	No	No	300.00
fixedSaving	DECIMAL (10,2)	Total fixed savings	No	No	1000.00
memberFund	DECIMAL (10,2)	Total member fund contribution	No	No	500.00
memberSaving	DECIMAL (10,2)	Total savings for the member	No	No	1500.00
alBai	DECIMAL (10, 2)	Total loan amount of Al-Bai	Yes	No	500.00
allnnah	DECIMAL (10, 2)	Total loan amount of Al-Innah	Yes	No	500.00
bPulihKenderaan	DECIMAL (10, 2)	Total loan amount of recover vehicle	Yes	No	500.00
roadTaxInsurance	DECIMAL (10, 2)	Total loan amount of road tax insurance	Yes	No	500.00
alQadrulHassan	DECIMAL (10, 2)	Total loan amount of Al-Qadrul Hassan	Yes	No	500.00
specialScheme	VARCHAR (100)	Name of any special scheme availed	Yes	No	KADA Loan Scheme
specialSeasonCarnival	DECIMAL (10, 2)	Special event funds	Yes	No	1500.00
dateUpdated	DATE	Last update date of the financial status	No	No	2024-07-23

transaction	transID (PK)	INT (20)	Unique ID for transaction	No	No	0009
	employeeID (PK)	VARCHAR (8)	Unique ID for member	No	No	0001
	transType	VARCHAR (50)	Type of transaction	No	No	Deposit
	transAmt	DECIMAL (12,2)	Transaction amount	No	No	2000.00
	transDate	DATE	Date of the transaction	No	No	2024-12-10
report	reportID (PK)	INT(20)	Unique ID for report	No	No	0002
	reportTtype	VARCHAR (50)	Type of the report	No	No	Financial
	dateGenerated	DATE	Date the report was generated	No	No	2024-12-10
officeAddress	officeAddress	VARCHAR (100)	Address of member's office	No	No	3A, Jalan Permai
	officePostcode	INT(6)	Postcode of office address	No	No	31400
	officeState	VARCHAR(30)	Office located state	No	No	Perak
homeAddress	homeAddress	VARCHAR (100)	Address of member's house	No	No	36, Jalan Indah Sakti
	homePostcode	INT(6)	Postcode of home address	No	No	31400
	officeState	VARCHAR(30)	Office located state	No	No	Perak

3. Logical Data Model (Logical ERD)

3.1 Logical Database Design (Logical ERD)



3.2 Data Dictionary for the Logical ERD

3.2.1 Entity: employee

Attribute Name	Type	Description
employeeID (PK)	VARCHAR (8)	Unique ID for the user
password	VARCHAR (30)	User's account password

3.2.2 Entity: admin

Attribute Name	Type	Description
employeeID (PK)	VARCHAR (8)	Unique ID for admin
staffName	VARCHAR (80)	Name of the admin

3.2.3 Entity: member

Attribute Name	Type	Description
employeeID (PK)	VARCHAR (8)	Unique ID for member
noPF	INT (4)	Provident fund of the member

memberName (FK)	VARCHAR (80)	Name of the member
ic	CHAR (12)	Identification number of the member
email (FK)	VARCHAR (30)	Member's email address
phoneNumber	VARCHAR (15)	The contact number of the member
sex	CHAR (1)	Gender of the member (M/F)
religion	VARCHAR (30)	The religion of the member
maritalStatus	VARCHAR (15)	Marital status
nation	VARCHAR (30)	Nationality of the member
position	VARCHAR (30)	Working position of the member
monthlySalary	DECIMAL (10,2)	Member's monthly salary
statementID	VARCHAR (20)	Unique ID for financial statement

3.2.4 Entity: memberRegistration

Attribute Name	Type	Description
memberRegistrationID (PK)	VARCHAR (20)	Unique ID for member registration

3.2.5 Entity: memberApplicationDetails

Attribute Name	Type	Description
regisDate	DATE	Date that member apply for loan
regisStatus	VARCHAR (8)	Status of the member registration

3.2.6 Entity: memberApproval

Attribute Name	Type	Description
regisApproveDate	DATE	Date when member registration is approved
regisStatus	VARCHAR (8)	Status of the member registration

3.2.7 Entity: loan

Attribute Name	Type	Description
loanApplicationID (PK)	VARCHAR (20)	Unique ID for loan application
amountRequested	DECIMAL (12,2)	Amount requested for the loan
financingPeriod	INT (2)	Loan repayment period in months
monthlyInstallments	DECIMAL (10,2)	Monthly repayment amount
bankNo	VARCHAR (15)	The bank number account

bankName	VARCHAR (50)	Bank name for loan transfer
financialType	VARCHAR (30)	Type of Financing

3.2.8 Entity: loanApproval

Attribute Name	Type	Description
loanApproveDate	DATE	Date that admin have approve the loan application
loanStatus	VARCHAR (20)	Status of the loan application

3.2.9 Entity: applicationDetails

Attribute Name	Type	Description
applicationDate	DATE	Date that member apply loan
loanStatus	VARCHAR (20)	Status of the loan application

3.2.10 Entity: guarantor

Attribute Name	Type	Description
icGuarantor (PK)	CHAR (12)	Identification number of the member
guarantorName	VARCHAR (80)	Name of the guarantor
memberIDGuarantor	VARCHAR (8)	Unique ID for member's guarantor
guarantorPFNo	INT (4)	Guarantor's provident fund number
telGuarantor	VARCHAR (15)	Contact number of the member' guarantor

3.2.11 Entity: financialStatus

Attribute Name	Type	Description
accountID (PK)	VARCHAR (8)	Unique ID for member's financial account
statementID	VARCHAR (20)	Unique ID for financial statement
memberSaving	DECIMAL (10,2)	Total savings for the member
alBai	DECIMAL (10, 2)	Total loan amount of Al-Bai
allnnah	DECIMAL (10, 2)	Total loan amount of Al-Innah
bPulihKenderaan	DECIMAL (10, 2)	Total loan amount of recover vehicle
roadTaxInsurance	DECIMAL (10, 2)	Total loan amount of road tax insurance
specialScheme	VARCHAR (100)	Name of any special scheme availed
alQadrulHassan	DECIMAL (10, 2)	Total loan amount of Al-Qadrul Hassan

dateUpdated	DATE	Last update date of the financial status

3.2.12 Entity: transaction

Attribute Name	Type	Description
transID (PK)	VARCHAR (20)	Unique ID for transaction
transType	VARCHAR (50)	Type of transaction
transAmt	DECIMAL (12,2)	Transaction amount
transDate	DATE	Date of the transaction

3.2.13 Entity: feesAndContribution

Attribute Name	Type	Description
entryFee	INT (3)	Fee for register as a member
modalShare	DECIMAL (10,2)	The member's share capital or investment in the organization.
feeCapital	DECIMAL (10,2)	Total fee capital of the member
deposit	DECIMAL (10,2)	Total amount deposited by member
contribution	DECIMAL (10,2)	The member’s recurring or voluntary financial contribution
fixedDeposit	DECIMAL (10,2)	Amount placed by the member as a fixed deposit with the organization.
others	VARCHAR (50)	Additional fees or contributions

3.2.14 Entity: familyMemberInfo

Attribute Name	Type	Description
icFamilyMember (PK)	CHAR (12)	Identification number of the family member
relationship	VARCHAR (30)	Relationship between family member and member
familyMemberName	VARCHAR (80)	Name of the family member

3.2.15 Entity: homeAddress

Attribute Name	Type	Description
homeAddress	VARCHAR (225)	Address for member’s home
homePostcode	VARCHAR (10)	Postcode for member’s home

homeState	VARCHAR (20)	State for member's home

3.2.16 Entity: officeAddress

Attribute Name	Type	Description
officeAddress	VARCHAR (225)	Address for member's office
officePostcode	VARCHAR (10)	Postcode for member's office
officeState	VARCHAR (20)	State for member's office

3.2.17 Entity: report

Attribute Name	Type	Description
reportType	VARCHAR (10)	Report type, either member, loan, monthly and yearly report
reportFormat	VARCHAR (3)	Report export/download format

3.2.18 Entity: reportDetails

Attribute Name	Type	Description
generateDate	DATE	Date report is generated
reportStatus	VARCHAR (11)	Status of the report, either active or non-active

3.3 Relational Database Schema(s)

EMPLOYEE (employeeID, password)
PK: employeeID

ADMIN (employeeID, staffName)
PK: employeeID

MEMBER(employeeID, noPF, memberName, ic, email, phoneNumber, sex, religion, maritalStatus, nation, position, monthlySalary, statementID)
PK: employeeID

MEMBERREGISTRATION (memberRegistrationID)
PK: memberRegistrationID

MEMBERAPPLICATIONDETAILS (employeeID, memberRegistrationID, regisDate, regisStatus)
PK: employeeID, memberRegistrationID
FK: employeeID references MEMBER(employeeID)
FK: memberRegistrationID references MEMBERREGISTRATION(memberRegistrationID)

MEMBERAPPROVAL(employeeID, memberRegistrationID, regisApproveDate, regisStatus)
PK: employeeID, memberRegistrationID
FK: employeeID references ADMIN(employeeID)
FK: memberRegistrationID references MEMBERREGISTRATION(memberRegistrationID)

LOAN(loanApplicationID, amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, financialType)
PK: loanApplicationID

LOANAPPROVAL(employeeID, loanApplicationID, loanApproveDate, loanStatus)
PK: employeeID, loanApplicationID
FK: employeeID references ADMIN(employeeID)
FK: loanApplicationID references LOAN(loanApplicationID)

APPLICATIONDETAILS (loanApplicationID, employeeID, applicationDate, loanStatus)
PK: loanApplicationID, employeeID
FK: loanApplicationID references LOAN(loanApplicationID)
FK: employeeID references MEMBER(employeeID)

GUARANTOR (icGuarantor, loanApplicationID, guarantorName, memberIDGuarantor, guarantorPFNo, telGuarantor)
PK: icGuarantor, loanApplicationID
FK: loanApplicationID references LOAN (loanApplicationID)

FINANCIALSTATUS (accountID, statementID, memberSaving, alBai, allnnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: accountID

TRANSACTION(transID, employeeID, accountID, transType, transAmt, transDate)
PK: transID, employeeID, accountID
FK: employeeID references MEMBER(employeeID)
FK: accountID references FINANCIALSTATUS(accountID)

FEESANDCONTRIBUTION (employeeID, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit, others)
PK: employeeID

FK: employeeID references MEMBER(employeeID)

FAMILYMEMBERINFO (<u>icFamilyMember</u>, <u>employeeID</u>, relationship, familyMemberName)
PK: icFamilyMember, employeeID
FK: employeeID references MEMBER(employeeID)

HOMEADDRESS (<u>employeeID</u>, homeAddress, homePostcode, homeState)
PK: employeeID
FK: employeeID references MEMBER(employeeID)

OFFICEADDRESS (<u>employeeID</u>, officeAddress, officePostcode, officeState)
PK: employeeID
FK: employeeID references MEMBER(employeeID)

REPORT(<u>employeeID</u>, <u>loanApplicationID</u>, <u>accountID</u>, reportType, reportFormat)
PK: employeeID, loanApplicationID, accountID
FK: employeeID references MEMBER(employeeID)
FK: loanApplicationID references LOAN(loanApplicationID)
FK: accountID references FINANCIALSTATUS(accountID)

REPORTDETAILS(<u>employeeID</u>, generateDate, reportStatus)
PK: employeeID
FK: employeeID references ADMIN(employeeID)

3.4 Normalization

1. <u>employee - admin</u>

1NF:
employee_admin (<u>employeeID</u>, password, staffName)
PK: employeeID

2NF:
employee (<u>employeeID</u>, password)
PK: employeeID
admin (<u>employeeID</u>, staffName)
PK: employeeID
FK: employeeID references employee(employeeID)

3NF:
employee (<u>employeeID</u>, password)

PK: employeeID
admin (employeeID, staffName)
PK: employeeID
FK: employeeID references employee(employeeID)

2. admin - loanApproval

1NF:
admin_loanApproval(employeeID, staffName, loanApproveDate, loanStatus)
PK: employeeID

2NF:
admin(employeeID, staffName)
PK: employeeID

loanApproval(employeeID, loanApproveDate, loanStatus)
PK: employeeID
FK: employeeID references admin(employeeID)

3NF:
admin(employeeID, staffName)
PK: employeeID

loanApproval(employeeID, loanApproveDate, loanStatus)
PK: employeeID
FK: employeeID references admin(employeeID)

3. loan - loanApproval

1NF:
loanApproval(loanApplicationID,amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, loanApproveDate, loanStatus)
PK: loanApplicationID

2NF:
loan(loanApplicationID,amountRequested, financingPeriod, monthlyInstallment)
PK: loanApplicationID
bank(bankNo, bankName)
PK: bankNo
loanApproval (loanApplicationID, bankNo, loanApproveDate, loanStatus)
PK: loanApplicationID, bankNo
FK: loanApplicationID reference loan(loanApplicationID)

FK: bankNo reference bank(bankNo)

3NF:
loan(<u>loanApplicationID</u>,amountRequested, financingPeriod, monthlyInstallment)
PK: loanApplicationID
bank(<u>bankNo</u>, bankName)
PK: bankNo
loanApproval (<u>loanApplicationID</u>, <u>bankNo,</u> loanApproveDate, loanStatus)
PK: loanApplicationID, bankNo
FK: loanApplicationID reference loan(loanApplicationID)
FK: bankNo reference bank(bankNo)

4. <u>loan - guarantorDetails</u>

1NF:
loan_guarantorDetails(<u>loanApplicationID</u>, <u>icGuarantor</u>, amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, guarantorName, memberIDGuarantor, telGuarantor)
PK: loanApplicationID, icGuarantor

2NF:
loan(<u>loanApplicationID</u>,amountRequested, financingPeriod, monthlyInstallment)
PK: loanApplicationID
guarantor(<u>icGuarantor</u>, guarantorName, guarantorPFNo, memberIDGuarantor, telGuarantor)
PK: icGuarator
loan_guarantorDetails(<u>loanApplicationID</u>, <u>icGuarantor</u>, bankNo)
PK: loanApplicationID, icGuarantor
FK: icGuarantor references guarantor (icGuarantor)
FK: loanApplicationID references loan (loanApplicationID)

3NF:
loan(<u>loanApplicationID</u>,amountRequested, financingPeriod, monthlyInstallment)
PK: loanApplicationID
guarantor(<u>icGuarantor</u>, guarantorName, guarantorPFNo, memberIDGuarantor, telGuarantor)
PK: icGuarator
loan_guarantorDetails(<u>loanApplicationID</u>, <u>icGuarantor</u>, bankNo)
PK: loanApplicationID, icGuarantor
FK: icGuarantor references guarantor (icGuarantor)
FK: loanApplicationID references loan (loanApplicationID)

5. <u>loan- applicationDetails</u>

1NF:
loan_Application (<u>loanApplicationID</u>,amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, loanApplicationDate, loanStatus)
PK: loanApplicationID

2NF:
Loan (<u>loanApplicationID</u>,amountRequested, financingPeriod, monthlyInstallments)
PK: loanApplicationID
bank (<u>bankNo</u>, bankName)
PK: bankNo
loanApplication (<u>loanApplicationID</u>, <u>bankNo</u>, loanApplicationDate, loanStatus)
PK: loanApplicationID, bankNo
FK: loanApplicationID reference loan(loanApplicationID)
FK: bankNo reference bank(bankNo)

3NF:
Loan (<u>loanApplicationID</u>,amountRequested, financingPeriod, monthlyInstallments)
PK: loanApplicationID
bank (<u>bankNo</u>, bankName)
PK: bankNo
loanApplication (<u>loanApplicationID</u>, <u>bankNo</u>, loanApplicationDate, loanStatus)
PK: loanApplicationID, bankNo
FK: loanApplicationID reference loan(loanApplicationID)
FK: bankNo reference bank(bankNo)

6. <u>admin - member</u>

1NF:
Admin_Member(<u>employeeID</u>, staffName, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID

2NF:
admin(<u>employeeID</u>, staffName)
PK: employeeID
member(<u>employeeID</u>, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
admin_member(<u>employeeID</u>)
PK: employeeID
FK: employeeID references admin(employeeID)
FK: employeeID references member(employeeID)

3NF:
admin(employeeID, staffName)
PK: employeeID
member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
admin_member(employeeID)
PK: employeeID
FK: employeeID references admin(employeeID)
FK: employeeID references member(employeeID)

7. employee - member

1NF:
employee_member(employeeID, password, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID

2NF:
employee(employeeID, password)
PK: employeeID
member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
employee_member(employeeID)
PK: employeeID
FK: employeeID references employee(employeeID)
FK: employeeID references member(employeeID)

3NF:
employee(employeeID, password)
PK: employeeID
member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
employee_member(employeeID)
PK: employeeID
FK: employeeID references employee(employeeID)
FK: employeeID references member(employeeID)

8. member - applicationDetails

1NF:
memberApplicationDetails(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, applicationDate, loanStatus)
PK: employeeID

2NF:
member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
applicationDetails (employeeID, applicationDate, loanStatus)
PK: employeeID
FK: employeeID references member(employeeID)

3NF:
member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
applicationDetails (employeeID, applicationDate, loanStatus)
PK: employeeID
FK: employeeID references member(employeeID)

9. member - transaction

1NF:
member_transaction (employeeID, transID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, transType, transAmt, transDate)
PK: employeeID, transID

2NF:
member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
transaction(transID,transType, transAmt,transDate)
PK: transID
Member_transaction (employeeID, transID)
PK: employeeID, transID
FK: employeeID references member(employeeID)
FK: transID references transaction(transID)

3NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

transaction(transID,transType, transAmt,transDate)

PK: transID

Member_transaction (employeeID, transID)

PK: employeeID, transID

FK: employeeID references member(employeeID)

FK: transID references transaction(transID)

10. member - financialStatus

1NF:

member_financialStatus(employeeID, accountID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: employeeID, accountID

2NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

member_financialStatus(employeeID, accountID)

PK: employeeID, accountID

FK: employee references member(employeeID)

FK: accountID references financialStatus(accountID)

3NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

member_financialStatus(employeeID, accountID)

PK: employeeID, accountID

FK: employee references member(employeeID)

FK: accountID references financialStatus(accountID)

11. transaction - financialStatus

1NF:
transaction_financialStatus (transID, accountID, transType, transAmt, transDate, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: transID, accountID

2NF:
transaction (transID, transType, transAmt, transDate)
PK: transID
financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: accountID
transaction_financialStatus (transID, accountID)
PK: transID, accountID
FK: transID references transaction(transID)
FK: accountID references financialStatus(accountID)

3NF:
transaction (transID, transType, transAmt, transDate)
PK: transID
financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: accountID
transaction_financialStatus (transID, accountID)
PK: transID, accountID
FK: transID references transaction(transID)
FK: accountID references financialStatus(accountID)

12. Member - homeAddress

1NF:
Member_homeAddress(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, homeAddress, homePostcode, homeState)
PK:employeeID

2NF:
Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID

member_homeAddress(employeeID, homeAddress, homePostcode, homeState)
PK: employeeID
FK: employeeID references member(employeeID)

3NF:
Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_homeAddress(employeeID, homeAddress, homePostcode, homeState)
PK: employeeID
FK: employeeID references member(employeeID)

13. Member - officeAddress

1NF:
Member_officeAddress(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, officeAddress, officePostcode, officeState)
PK:employeeID

2NF:
Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_officeAddress(employeeID, officeAddress, officePostcode, officeState)
PK: employeeID
FK: employeeID references member(employeeID)

3NF:
Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_officeAddress(employeeID, officeAddress, officePostcode, officeState)
PK: employeeID
FK: employeeID references member(employeeID)

14. Admin - memberApproval

1NF:
admin_memberApproval(employeeID, staffName, regisApproveDate, regStatus)
PK: employeeID

2NF:
admin_memberApproval(<u>employeeID</u>, staffName, regisApproveDate, regStatus)
PK: employeeID

3NF:
admin_memberApproval(<u>employeeID</u>, staffName, regisApproveDate, regStatus)
PK: employeeID

15. <u>memberApproval - memberRegistration</u>

1NF:
memberApproval_memberRegistration(<u>memberRegistrationID</u>, regisApproveDate, regisStatus)
PK: memberRegistrationID

2NF:
memberApproval_memberRegistration(<u>memberRegistrationID</u>, regisApproveDate, regisStatus)
PK: memberRegistrationID

3NF:
memberApproval_memberRegistration(<u>memberRegistrationID</u>, regisApproveDate, regisStatus)
PK: memberRegistrationID

16. <u>memberRegistration - familyMemberInfo</u>

1NF:
memberRegistration_familyMemberInfo(<u>memberRegistrationID</u>, <u>icFamilyMember</u>, relationship, familyMemberName)
PK: memberRegistrationID, icFamilyMember

2NF:
familyMemberInfo(<u>icFamilyMember</u>, relationship, familyMemberName)
PK: icFamilyMember
memberRegistration_familyMemberInfo(<u>memberRegistrationID</u>, <u>icFamilyMember</u>)
PK: memberRegistrationID, icFamilyMember
FK: icFamilyMember references familyMemberInfo(icFamilyMember)

3NF:

familyMemberInfo(<u>icFamilyMember</u>, relationship, familyMemberName) employeeID
PK: icFamilyMember
memberRegistration_familyMemberInfo(<u>memberRegistrationID</u>, <u>icFamilyMember</u>)
PK: memberRegistrationID, icFamilyMember
FK: icFamilyMember references familyMemberInfo(icFamilyMember)

17. <u>memberRegistration - feesAndContribution</u>

1NF:
memberRegistration_feesAndContribution(<u>memberRegistrationID</u>, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit, others)
PK: memberRegistrationID

2NF:
memberRegistration_feesAndContribution(<u>memberRegistrationID</u>, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit, others)
PK: memberRegistrationID

3NF:
memberRegistration_feesAndContribution(<u>memberRegistrationID</u>, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit, others)
PK: memberRegistrationID

18. <u>memberRegistration - memberApplicationDetails</u>

1NF:
memberRegistration_memberApplicationDetails(<u>memberRegistrationID</u>, regisDate, regisStatus)
PK: memberRegistrationID

2NF:
memberRegistration_memberApplicationDetails(<u>memberRegistrationID</u>, regisDate, regisStatus)
PK: memberRegistrationID

3NF:
memberRegistration_memberApplicationDetails(<u>memberRegistrationID</u>, regisDate, regisStatus)
PK: memberRegistrationID

19. Member - memberApplicationDetails

1NF:
member_memberApplicationDetails(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, regisDate, regisStatus)
PK: employeeID

2NF:
member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_memberApplicationDetails(employeeID,regisDate, regisStatus)
PK: employeeID
FK: employeeID references member(employeeID)

3NF:
member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_memberApplicationDetails(employeeID,regisDate, regisStatus)
PK: employeeID
FK: employeeID references member(employeeID)

20. admin_reportDetails

1NF:
admin_reportDetails (employeeID, staffName, generateDate, reportStatus)
PK: employeeID

2NF:
admin (employeeID, staffName)
PK: employeeID
admin_reportDetails (employeeID, generateDate, reportStatus)
PK: employeeID
FK: employeeID references admin(employeeID)

3NF:
admin (employeeID, staffName)
PK: employeeID
admin_reportDetails (employeeID, generateDate, reportStatus)
PK: employeeID
FK: employeeID references admin(employeeID)

21. member_report

1NF:
member_report (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, reportType, reportFormat)
PK: employeeID

2NF:
member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_report (employeeID, regisDate, regisStatus)
PK: employeeID
FK: employeeID references member(employeeID)

3NF:
member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)
PK: employeeID
member_report (employeeID, regisDate, regisStatus)
PK: employeeID
FK: employeeID references member(employeeID)

22. financialStatus_report

1NF:
financialStatus_report (accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated, reportType, reportFormat)
PK: accountID

2NF:
financialStatus (accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: accountID
financialStatus_report (accountID, regisDate, regisStatus)
PK: accountID
FK: accountID references financialStatus(accountID)

3NF:
financialStatus (accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: accountID

financialStatus_report (accountID, regisDate, regisStatus)
PK: accountID
FK: accountID references financialStatus(accountID)

23. report_reportDetails

1NF:
report_reportDetails (reportType, reportFormat, generateDate, reportStatus)
PK: reportType, generateDate

2NF:
report_reportDetails (reportType, reportFormat, generateDate, reportStatus)
PK: reportType, generateDate

3NF:
report_reportDetails (reportType, reportFormat, generateDate, reportStatus)
PK: reportType, generateDate

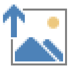
4.0 Interface Design and SQL Implementation

4.1 Interface Design

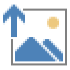
The interface includes a membership application, loan application, profile page, view financial status, reporting and status page.

4.1.1 Login Interface

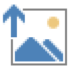
Login interface for both member and admin

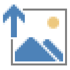


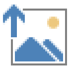
If you want to apply membership for koperasi kada and dont have an account yet at our system, you can apply an account

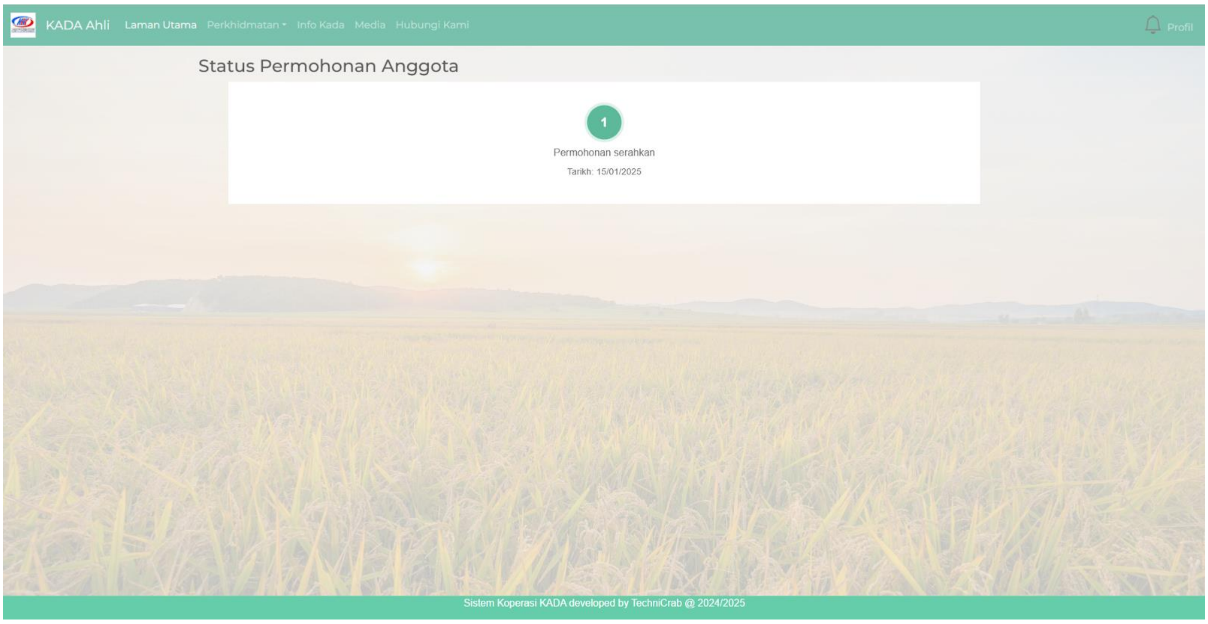


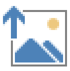
After apply an account and login, you can now apply the membership





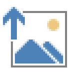






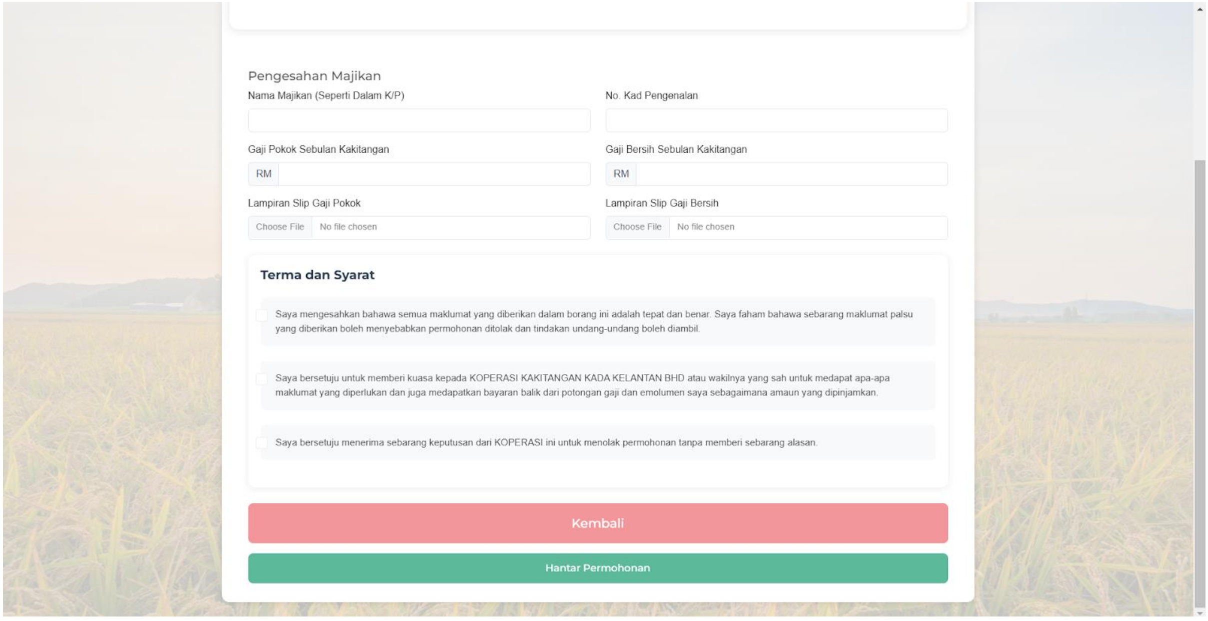
4.1.2 Member Interface

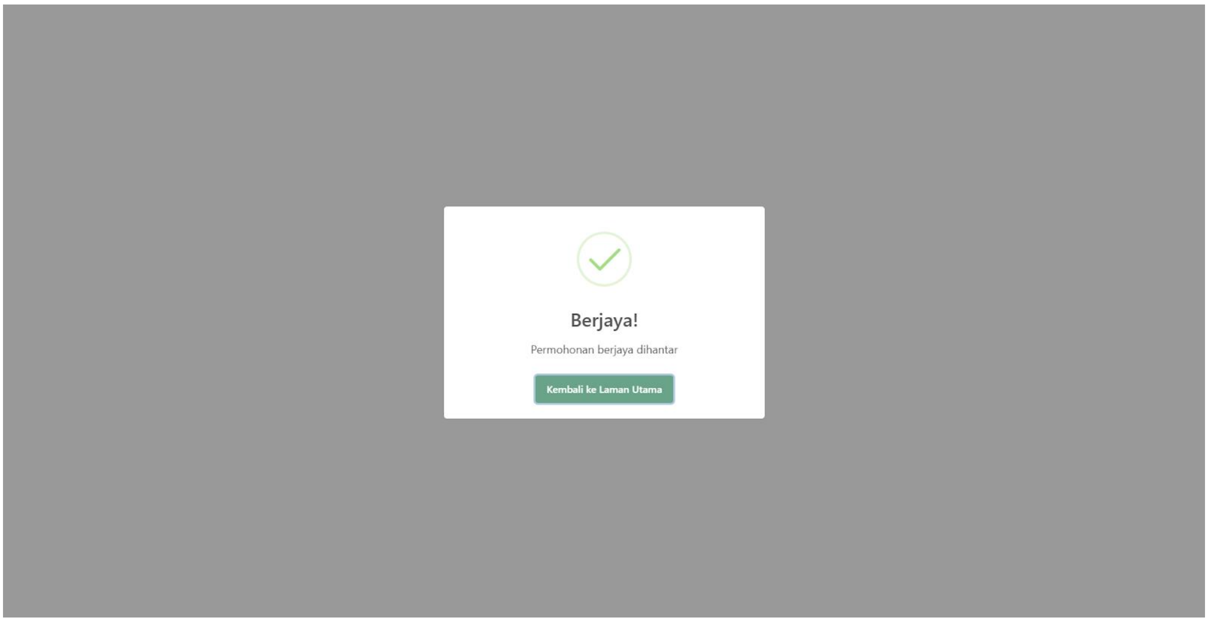
Main Page



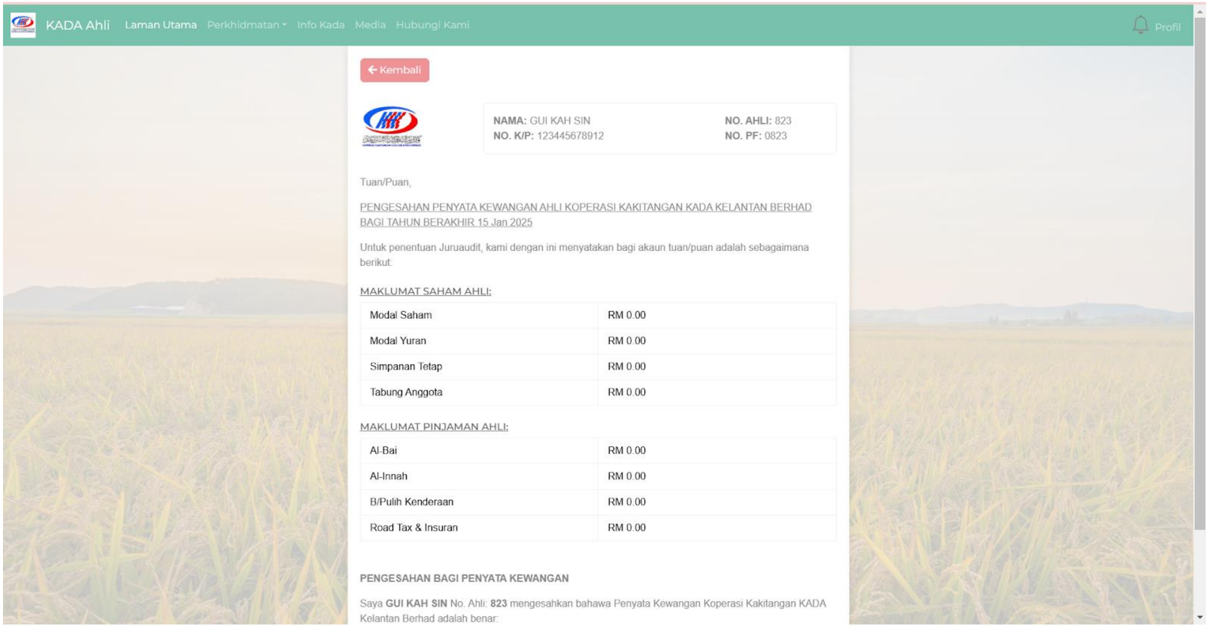
Profile Page

Loan Application





Financial status page



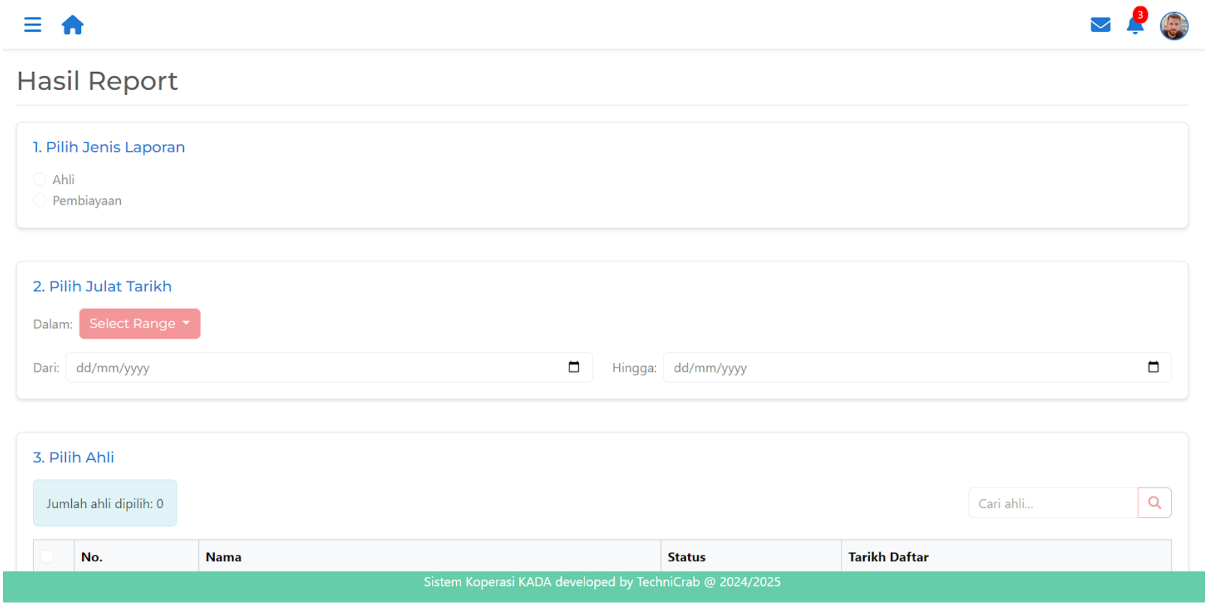
4.1.3 Admin Interface

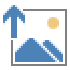
Main Page

Member Application

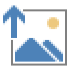
Loan Application

Generate Report





View Report



4.2 SQL Statements (DDL & DML)

1. Apply membership & View membership application status

```
CREATE TABLE `tb_member` (
  `employeeID` int(4) NOT NULL,
  `memberName` varchar(255) NOT NULL,
  `email` varchar(255) DEFAULT NULL COMMENT 'Email address',
  `ic` varchar(20) NOT NULL,
  `maritalStatus` varchar(20) NOT NULL,
  `sex` varchar(10) DEFAULT NULL,
  `religion` varchar(50) DEFAULT NULL,
```

```
  `nation` varchar(50) DEFAULT NULL,
  `no_pf` varchar(50) DEFAULT NULL,
  `position` varchar(100) DEFAULT NULL COMMENT 'Job position',
  `phoneNumber` varchar(20) DEFAULT NULL COMMENT 'Primary contact number',
  `phoneHome` varchar(20) DEFAULT NULL COMMENT 'Home contact number',
  `monthlySalary` decimal(10,2) DEFAULT NULL COMMENT 'Monthly salary amount',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `tb_member` (`employeeID`, `memberName`, `email`, `ic`, `maritalStatus`, `sex`,
`religion`, `nation`, `no_pf`, `position`, `phoneNumber`, `phoneHome`, `monthlySalary`, `created_at`,
`updated_at`) VALUES
(322, 'Cheryl Cheong Kah Voon', 'cheryl@gmail.com', '040322040352', 'Bujang', 'Perempuan',
'Buddha', 'Cina', '322', '322', '1121192282', '035331234', 2000.00, '2025-01-14 11:18:24', '2025-01-14
11:19:10') ;

ALTER TABLE `tb_member`
  ADD PRIMARY KEY (`employeeID`);

CREATE TABLE `tb_memberregistration_familymemberinfo` (
  `employeeID` int(4) NOT NULL,
  `relationship` varchar(50) NOT NULL,
  `name` varchar(100) NOT NULL,
  `icFamilyMember` varchar(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `tb_memberregistration_familymemberinfo` (`employeeID`, `relationship`, `name`,
`icFamilyMember`) VALUES
(322, 'Anak', 'mini cheryl', '252222-22-2222') ;

ALTER TABLE `tb_memberregistration_familymemberinfo`
  ADD PRIMARY KEY (`employeeID`,`icFamilyMember`);

ALTER TABLE `tb_memberregistration_familymemberinfo`
  ADD CONSTRAINT `tb_memberregistration_familymemberinfo_ibfk_1` FOREIGN KEY
(`employeeID`) REFERENCES `tb_member` (`employeeID`);


CREATE TABLE `tb_memberregistration_feesandcontribution` (
  `employeeID` int(4) NOT NULL,
  `entryFee` int(11) NOT NULL DEFAULT 0,
  `modalShare` int(11) NOT NULL DEFAULT 0,
  `feeCapital` int(11) NOT NULL DEFAULT 0,
  `deposit` int(11) NOT NULL DEFAULT 0,
  `contribution` int(11) NOT NULL DEFAULT 0,
  `fixedDeposit` int(11) NOT NULL DEFAULT 0,
  `others` int(11) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

```
INSERT INTO `tb_memberregistration_feesandcontribution` (`employeeID`, `entryFee`,
`modalShare`, `feeCapital`, `deposit`, `contribution`, `fixedDeposit`, `others`) VALUES
(322, 100, 1000, 0, 0, 600, 1200, 0) ;

ALTER TABLE `tb_memberregistration_feesandcontribution`
  ADD PRIMARY KEY (`employeeID`);

ALTER TABLE `tb_memberregistration_feesandcontribution`
  ADD CONSTRAINT `fk_member_fees` FOREIGN KEY (`employeeID`) REFERENCES
`tb_member` (`employeeID`) ON DELETE CASCADE ON UPDATE CASCADE;

CREATE TABLE `tb_member_homeaddress` (
  `employeeID` int(4) NOT NULL,
  `homeAddress` varchar(255) NOT NULL,
  `homePostcode` varchar(10) NOT NULL,
  `homeState` varchar(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `tb_member_homeaddress` (`employeeID`, `homeAddress`, `homePostcode`,
`homeState`) VALUES
(322, 'M19', '82000', 'Melaka') ;

ALTER TABLE `tb_member_homeaddress`
  ADD PRIMARY KEY (`employeeID`);

ALTER TABLE `tb_member_homeaddress`
  ADD CONSTRAINT `tb_member_homeaddress_ibfk_1` FOREIGN KEY (`employeeID`)
REFERENCES `tb_member` (`employeeID`);

CREATE TABLE `tb_member_officeaddress` (
  `employeeID` int(4) NOT NULL,
  `officeAddress` varchar(255) NOT NULL,
  `officePostcode` varchar(10) NOT NULL,
  `officeState` varchar(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `tb_member_officeaddress` (`employeeID`, `officeAddress`, `officePostcode`,
`officeState`) VALUES
(322, 'KTDI', '81310', 'Melaka'),

ALTER TABLE `tb_member_officeaddress`
  ADD PRIMARY KEY (`employeeID`);

ALTER TABLE `tb_member_officeaddress`
  ADD CONSTRAINT `tb_member_officeaddress_ibfk_1` FOREIGN KEY (`employeeID`)
REFERENCES `tb_member` (`employeeID`);
COMMIT;

SELECT employeeID FROM tb_member WHERE employeeID = ?;

INSERT INTO tb_member (
    employeeID,
```

```
    memberName,
    email,
    ic,
    maritalStatus,
    sex,
    religion,
    nation,
    no_pf,
    position,
    phoneNumber,
    phoneHome,
    monthlySalary
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);


INSERT INTO tb_member_homeaddress (employeeID, homeAddress, homePostcode, homeState)
VALUES (?, ?, ?, ?);

INSERT INTO tb_member_officeaddress (employeeID, officeAddress, officePostcode, officeState)
VALUES (?, ?, ?, ?);


INSERT INTO tb_memberregistration_feesandcontribution
(employeeID, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit)
VALUES (?, ?, ?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
entryFee = VALUES(entryFee),
modalShare = VALUES(modalShare),
feeCapital = VALUES(feeCapital),
deposit = VALUES(deposit),
contribution = VALUES(contribution),
fixedDeposit = VALUES(fixedDeposit);


DELETE FROM tb_memberregistration_familymemberinfo WHERE employeeID = ?;

INSERT INTO tb_memberregistration_familymemberinfo
(employeeID, relationship, name, icFamilyMember)
VALUES (?, ?, ?, ?);

SELECT
    CASE
        WHEN mr.memberRegistrationID IS NOT NULL AND mr.regisStatus = 'Diluluskan' THEN
'Diluluskan'
        WHEN mr.memberRegistrationID IS NOT NULL AND mr.regisStatus = 'Ditolak' THEN
'Ditolak'
        WHEN mr.memberRegistrationID IS NOT NULL THEN 'Belum Selesai'
        ELSE 'Tiada Rekod'
    END as regisStatus,
    CASE
        WHEN mr.regisDate IS NOT NULL THEN DATE(mr.regisDate)
        WHEN m.created_at IS NOT NULL THEN DATE(m.created_at)
        ELSE CURRENT_DATE()
```

```
   END as regisDate
FROM tb_member m
LEFT JOIN tb_memberregistration_memberapplicationdetails mr
ON m.employeeID = mr.memberRegistrationID
WHERE m.employeeID = ?;
```

2. Apply for loan & View loan application status

```
CREATE TABLE `tb_bank` (
`bankID` int(11) NOT NULL,
`loanApplicationID` int(11) NOT NULL,
`employeeID` int(4) NOT NULL,
`bankName` varchar(100) NOT NULL,
`accountNo` varchar(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `tb_bank` (`bankID`, `loanApplicationID`, `employeeID`, `bankName`,
`accountNo`) VALUES
(28, 36, 322, 'Affin Bank', '1234'),

ALTER TABLE `tb_bank`
  ADD PRIMARY KEY (`bankID`),
  ADD KEY `loanApplicationID` (`loanApplicationID`);

ALTER TABLE `tb_bank`
  MODIFY `bankID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=35;

ALTER TABLE `tb_bank`
  ADD CONSTRAINT `tb_bank_ibfk_1` FOREIGN KEY (`loanApplicationID`) REFERENCES
`tb_loanapplication` (`loanApplicationID`);


CREATE TABLE `tb_guarantor` (
  `guarantorID` int(11) NOT NULL,
  `loanApplicationID` int(11) DEFAULT NULL,
  `employeeID` int(11) DEFAULT NULL,
  `guarantorName` varchar(255) NOT NULL,
  `guarantorIC` varchar(20) NOT NULL,
  `guarantorPhone` varchar(20) NOT NULL,
  `guarantorPFNo` varchar(20) NOT NULL,
  `guarantorMemberNo` varchar(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;


INSERT INTO `tb_guarantor` (`guarantorID`, `loanApplicationID`, `employeeID`,
`guarantorName`, `guarantorIC`, `guarantorPhone`, `guarantorPFNo`, `guarantorMemberNo`)
VALUES
```

```
(5, 36, 322, 'CHERYL', '111111111111', '1111111111', '111', '1111'),

ALTER TABLE `tb_guarantor`
  ADD PRIMARY KEY (`guarantorID`);

ALTER TABLE `tb_guarantor`
  MODIFY `guarantorID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=17;



CREATE TABLE `tb_loan` (
  `loanType` varchar(30) NOT NULL,
  `loanID` int(11) NOT NULL,
  `loanApplicationID` int(11) NOT NULL,
  `employeeID` int(4) NOT NULL,
  `amountRequested` decimal(10,2) NOT NULL COMMENT 'amount that want to pinjam',
  `financingPeriod` int(11) NOT NULL,
  `monthlyInstallments` decimal(10,2) NOT NULL,
  `employerName` varchar(100) NOT NULL,
  `employerIC` varchar(14) NOT NULL,
  `basicSalary` decimal(10,2) NOT NULL,
  `netSalary` decimal(10,2) NOT NULL,
  `basicSalaryFile` varchar(255) NOT NULL,
  `netSalaryFile` varchar(255) NOT NULL,
  `balance` decimal(10,2) DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

INSERT INTO `tb_loan` (`loanType`, `loanID`, `loanApplicationID`, `employeeID`,
`amountRequested`, `financingPeriod`, `monthlyInstallments`, `employerName`, `employerIC`,
`basicSalary`, `netSalary`, `basicSalaryFile`, `netSalaryFile`, `balance`, `created_at`, `updated_at`)
VALUES
('AL-BAI', 20, 36, 322, 3000.00, 12, 250.00, 'NAMA MAJIKAN', '222222222222', 2000.00,
1500.00, 'uploads/6786a2674a8b4_MAFEST \'25 UNIT HADIAH (2).pdf',
'uploads/6786a2674aa3b_OPERA \'25 UNIT HADIAH (2).pdf', NULL, '2025-01-18 16:46:10',
'2025-01-18 16:47:01'),

ALTER TABLE `tb_loan`
  ADD PRIMARY KEY (`loanID`),
  ADD KEY `loanApplicationID` (`loanApplicationID`);

ALTER TABLE `tb_loan`
  MODIFY `loanID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=26;

ALTER TABLE `tb_loan`
  ADD CONSTRAINT `tb_loan_ibfk_1` FOREIGN KEY (`loanApplicationID`) REFERENCES
`tb_loanapplication` (`loanApplicationID`);

ALTER TABLE `tb_loan`
  DROP `basicSalaryFile`;
```

```
CREATE TABLE `tb_loanapplication` (
  `loanApplicationID` int(11) NOT NULL,
  `employeeID` int(4) NOT NULL,
  `loanApplicationDate` date NOT NULL,
  `loanStatus` varchar(20) DEFAULT 'Pending',
  `amountRequested` decimal(10,2) NOT NULL,
  `financingPeriod` int(11) NOT NULL,
  `monthlyInstallments` decimal(10,2) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;


INSERT INTO `tb_loanapplication` (`loanApplicationID`, `employeeID`, `loanApplicationDate`,
`loanStatus`, `amountRequested`, `financingPeriod`, `monthlyInstallments`) VALUES
(36, 322, '2025-01-15', 'Diluluskan', 3000.00, 12, 250.00),

ALTER TABLE `tb_loanapplication`
  ADD PRIMARY KEY (`loanApplicationID`);

ALTER TABLE `tb_loanapplication`
  MODIFY `loanApplicationID` int(11) NOT NULL AUTO_INCREMENT,
AUTO_INCREMENT=43;


-- Check duplicate loan applications
SELECT COUNT(*)
FROM tb_loanapplication
WHERE employeeID = ?
AND loanStatus IN ('Dalam Proses', 'Diluluskan')
AND loanType = ?


-- Check registration status
SELECT COALESCE(mr.regisStatus, 'Belum Selesai') as regisStatus
FROM tb_member m
LEFT JOIN tb_memberregistration_memberapplicationdetails mr
ON m.employeeID = mr.memberRegistrationID
WHERE m.employeeID = ?


-- Get member details
SELECT m.*, h.homeAddress, h.homePostcode, h.homeState,
       o.officeAddress, o.officePostcode, o.officeState
FROM tb_member m
LEFT JOIN tb_member_homeaddress h ON m.employeeID = h.employeeID
LEFT JOIN tb_member_officeaddress o ON m.employeeID = o.employeeID
WHERE m.employeeID = ?


-- Get latest loan application
SELECT loanApplicationID FROM tb_loanapplication
WHERE employeeID = ?
```

```
ORDER BY created_at DESC LIMIT 1


-- Update loan status
UPDATE tb_loanapplication
SET loanStatus = 'Dalam Proses'
WHERE loanApplicationID = ?


-- Update loan balance
UPDATE tb_loan
SET balance = amountRequested
WHERE loanApplicationID = ?


SELECT memberName
FROM tb_member
WHERE employeeID = ?
 SELECT
    la.loanApplicationID,
    la.loanApplicationDate,
    la.loanStatus,
    la.amountRequested,
    l.loanType,
    l.financingPeriod,
    l.monthlyInstallments,
    l.balance
FROM tb_loanapplication la
LEFT JOIN tb_loan l ON l.loanApplicationID = la.loanApplicationID
WHERE la.employeeID = ?
ORDER BY la.loanApplicationID DESC
```

3. Approve member registration

```
SELECT
   m.employeeID as memberRegistrationID,
   m.memberName,
   m.ic,
   m.created_at as regisDate,
   COALESCE(
       (SELECT regisStatus
        FROM tb_memberregistration_memberapplicationdetails
        WHERE memberRegistrationID = m.employeeID
        ORDER BY regisDate DESC
        LIMIT 1),
```

```
        'Belum Selesai'
    ) as regisStatus
FROM
    tb_member m
LEFT JOIN tb_memberregistration_memberapplicationdetails md
    ON m.employeeID = md.memberRegistrationID
GROUP BY
    m.employeeID, m.memberName, m.ic, m.created_at;


SELECT memberRegistrationID
FROM tb_memberregistration_memberapplicationdetails
WHERE memberRegistrationID = ?


UPDATE tb_memberregistration_memberapplicationdetails
SET regisStatus = ?, regisDate = ?
WHERE memberRegistrationID = ?


INSERT INTO tb_memberregistration_memberapplicationdetails
(memberRegistrationID, regisDate, regisStatus)
VALUES (?, ?, ?)


SELECT
    m.*,
    COALESCE(mha.homeAddress, '') as homeAddress,
    COALESCE(mha.homePostcode, '') as homePostcode,
    COALESCE(mha.homeState, '') as homeState,
    COALESCE(moa.officeAddress, '') as officeAddress,
    COALESCE(moa.officePostcode, '') as officePostcode,
    COALESCE(moa.officeState, '') as officeState,
    COALESCE(mfc.entryFee, 0) as entryFee,
    COALESCE(mfc.modalShare, 0) as modalShare,
    COALESCE(mfc.feeCapital, 0) as feeCapital,
    COALESCE(mfc.deposit, 0) as deposit,
    COALESCE(mfc.contribution, 0) as contribution,
    COALESCE(mfc.fixedDeposit, 0) as fixedDeposit,
    COALESCE(mfc.others, 0) as others
FROM tb_member m
LEFT JOIN tb_member_homeaddress mha ON m.employeeID = mha.employeeID
LEFT JOIN tb_member_officeaddress moa ON m.employeeID = moa.employeeID
LEFT JOIN tb_memberregistration_feesandcontribution mfc ON m.employeeID = mfc.employeeID
WHERE m.employeeID = ?


SELECT *
FROM tb_memberregistration_familymemberinfo
WHERE employeeID = ?

```

4. Approve loan application

```
SELECT
    la.loanApplicationID,
    m.memberName,
    m.ic,
    la.loanApplicationDate as applicationDate,
    COALESCE(la.loanStatus, 'Belum Selesai') as loanStatus
FROM
    tb_loanapplication la
JOIN
    tb_member m ON la.employeeID = m.employeeID
ORDER BY
    la.loanApplicationDate DESC;


UPDATE tb_loanapplication
SET loanStatus = ?,
    loanApplicationDate = CASE
        WHEN loanStatus != ? THEN CURRENT_DATE
        ELSE loanApplicationDate
    END
WHERE loanApplicationID = ?


SELECT
    la.loanApplicationID,
    la.amountRequested,
    la.financingPeriod,
    la.monthlyInstallments,
    l.loanType,
    l.employerName,
    l.employerIC,
    l.basicSalary,
    l.netSalary,
    m.memberName,
    m.ic,
    m.maritalStatus,
    m.sex,
    m.religion,
    m.nation,
    m.no_pf,
    m.phoneNumber,
    mha.homeAddress,
    mha.homePostcode,
    mha.homeState,
    moa.officeAddress,
    moa.officePostcode,
    moa.officeState,
    b.bankName,
```

```
    b.accountNo,
    l.basicSalaryFile,
    l.netSalaryFile
FROM tb_loanapplication la
LEFT JOIN tb_loan l ON l.loanApplicationID = la.loanApplicationID
LEFT JOIN tb_member m ON la.employeeID = m.employeeID
LEFT JOIN tb_member_homeaddress mha ON m.employeeID = mha.employeeID
LEFT JOIN tb_member_officeaddress moa ON m.employeeID = moa.employeeID
LEFT JOIN tb_bank b ON b.loanApplicationID = la.loanApplicationID
WHERE la.loanApplicationID = ?


SELECT basicSalary, netSalary FROM tb_loan WHERE loanApplicationID = ?


SELECT * FROM tb_guarantor WHERE loanApplicationID = ?

```

5. Manage Pprofile

```
//Profile which the member already applies a loan
SELECT m.*,
            h.homeAddress, h.homePostcode, h.homeState,
            o.officeAddress, o.officePostcode, o.officeState
      FROM tb_member m
      LEFT JOIN tb_member_homeaddress h ON m.employeeID = h.employeeID
      LEFT JOIN tb_member_officeaddress o ON m.employeeID = o.employeeID
      WHERE m.employeeID = ?


//Profile which the user hasn't applied for membership and member hasn't apply for
a loan
if ($isMember) {
    $sql = "SELECT m.*,
                h.homeAddress, h.homePostcode, h.homeState,
                o.officeAddress, o.officePostcode, o.officeState
          FROM tb_member m
          LEFT JOIN tb_member_homeaddress h ON m.employeeID = h.employeeID
          LEFT JOIN tb_member_officeaddress o ON m.employeeID =
o.employeeID
          WHERE m.employeeID = ?";
} else {
    $sql = "SELECT e.*
          FROM tb_employee e
```

```
            WHERE e.employeeID = ?";
}

//Update profile page
UPDATE tb_member SET
                memberName = ?,
                email = ?,
                ic = ?,
                maritalStatus = ?,
                sex = ?,
                religion = ?,
                nation = ?,
                phoneNumber = ?
                WHERE employeeID = ?

//update home address
UPDATE tb_member_homeaddress SET
              homeAddress = ?,
 homePostcode = ?,
             homeState = ?
             WHERE employeeID = ?

//update office address
UPDATE tb_member_officeaddress SET
                officeAddress = ?,
                officePostcode = ?,
                officeState = ?
                WHERE employeeID = ?
```

6. View financial status

```
CREATE TABLE `tb_transaction` (
  `employeeID` int(4) NOT NULL,
  `transType` varchar(10) NOT NULL,
  `transAmt` int(6) NOT NULL,
  `transDate` date NOT NULL,
  `transID` int(10) NOT NULL
 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;

 INSERT INTO `tb_transaction` (`employeeID`, `transType`, `transAmt`,
`transDate`,  `transID`) VALUES
 (4567, 'Simpanan-M', 300, '2025-01-19', 53),
 (4567, 'Simpanan-Y', 40, '2025-01-19', 54),
 (4567, 'Simpanan-S', 40, '2025-01-19', 55),
 (4567, 'Simpanan-T', 40, '2025-01-19', 56),
 (4567, 'Bayaran Ba', 750, '2025-01-19', 57),
 (322, 'Simpanan-M', 1000, '2025-01-03', 58),
 (322, 'Simpanan-S', 1200, '2025-01-03', 59),
 (322, 'Simpanan-T', 600, '2025-01-03', 60),
 (322, 'Simpanan-M', 1000, '2025-01-03', 61),
 (322, 'Simpanan-S', 1200, '2025-01-03', 62),
 (322, 'Simpanan-T', 600, '2025-01-03', 63),
 (322, 'Bayaran Ba', 250, '2025-01-11', 64),
 (322, 'Bayaran Ba', 250, '2025-01-11', 65),
 (322, 'Bayaran Ba', 250, '2025-01-10', 66),
 (9876, 'Entry Fee', 20, '2025-01-19', 80),
 (9876, 'Deposit', 20, '2025-01-19', 81),
 (9876, 'Simpanan-M', 300, '2025-01-19', 82),
 (9876, 'Simpanan-Y', 20, '2025-01-19', 83),
 (9876, 'Simpanan-S', 20, '2025-01-19', 84),
 (9876, 'Simpanan-T', 20, '2025-01-19', 85),
 (9876, 'Entry Fee', 20, '2025-02-19', 86),
 (9876, 'Deposit', 20, '2025-02-19', 87),
 (9876, 'Simpanan-M', 300, '2025-02-19', 88),
 (9876, 'Simpanan-Y', 20, '2025-02-19', 89),
 (9876, 'Simpanan-S', 20, '2025-02-19', 90),
 (9876, 'Simpanan-T', 20, '2025-02-19', 91),
 (9876, 'Entry Fee', 20, '2025-03-19', 92),
 (9876, 'Deposit', 20, '2025-03-19', 93),
 (9876, 'Simpanan-M', 300, '2025-03-19', 94),
 (9876, 'Simpanan-Y', 20, '2025-03-19', 95),
 (9876, 'Simpanan-S', 20, '2025-03-19', 96),
 (9876, 'Simpanan-T', 20, '2025-03-19', 97),
```

```
 (9876, 'Bayaran Ba', 250, '2025-01-11', 98),
 (9876, 'Bayaran Ba', 250, '2025-02-11', 99),
 (9876, 'Bayaran Ba', 250, '2025-03-11', 101);

 CREATE TABLE `tb_financialstatus` (
  `employeeID` int(4) NOT NULL,
  `modalShare` int(11) NOT NULL,
  `feeCapital` int(11) NOT NULL,
  `contribution` int(11) NOT NULL,
  `fixedDeposit` int(11) NOT NULL,
  `dateUpdated` int(10) NOT NULL
 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;

 INSERT INTO `tb_financialstatus` (`employeeID`, `modalShare`, `feeCapital`,
`contribution`, `fixedDeposit`, `dateUpdated`) VALUES
 (322, 0, 0, 0, 0, 2147483647),
 (1234, 0, 0, 0, 0, 2147483647),
 (5522, 0, 0, 0, 0, 2147483647),
 (9876, 900, 60, 60, 60, 1737350531);



//fetch financial data
SELECT * FROM tb_memberregistration_feesandcontribution
        WHERE employeeID = ?

//get the transaction amount
SELECT transType, transAmt
              FROM tb_transaction
              WHERE employeeID = ?

//get fee distribution
SELECT
    entryFee,
    deposit,
    modalShare,
    feeCapital,
    contribution,
    fixedDeposit
FROM tb_memberregistration_feesandcontribution
WHERE employeeID = ?

//get all transaction
```

```
SELECT transAmt, transDate
            FROM tb_transaction
            WHERE employeeID = ?
            ORDER BY transDate ASC


//fatech bank no
SELECT accountNo FROM tb_bank WHERE employeeID = ? ORDER BY
bankID DESC LIMIT 1

//update balance loan
UPDATE tb_loan l
JOIN tb_loanapplication la ON l.employeeID = la.employeeID
SET l.balance = CASE
   WHEN l.balance IS NULL THEN la.amountRequested
   ELSE l.balance
END
WHERE l.employeeID = ?
AND la.loanStatus = 'Diluluskan'


//fetch amount already paid
SELECT SUM(transAmt) as total_paid
FROM tb_transaction
WHERE employeeID = ?
AND transType = 'Bayaran Ba'

//fetch loan data
SELECT
   l.loanType,
   l.balance,
   la.amountRequested,
   la.monthlyInstallments
FROM tb_loan l
JOIN tb_loanapplication la ON l.employeeID = la.employeeID
WHERE l.employeeID = ?
AND la.loanStatus = 'Diluluskan'

//update balance in tb_loan
UPDATE tb_loan
               SET balance = ?
               WHERE employeeID = ?
               AND loanType = ?
```

```
//fetch latest amount
SELECT
    modalShare,
    feeCapital,
    fixedDeposit,
    contribution
FROM tb_financialstatus
WHERE employeeID = ?

//update tb_financialStatus
INSERT INTO tb_financialstatus
                (employeeID, modalShare, feeCapital, contribution, fixedDeposit,
dateUpdated)
                VALUES (?, ?, ?, ?, ?, UNIX_TIMESTAMP())
                ON DUPLICATE KEY UPDATE
                modalShare = VALUES(modalShare),
                feeCapital = VALUES(feeCapital),
                contribution = VALUES(contribution),
                fixedDeposit = VALUES(fixedDeposit),
                dateUpdated = UNIX_TIMESTAMP()


//fetch amount for each loan type
SELECT
    l.loanType,
    l.balance
FROM tb_loan l
JOIN tb_loanapplication la ON l.employeeID = la.employeeID
WHERE l.employeeID = ?
AND la.loanStatus = 'Diluluskan'

//view financial statement
//fetch member detail
SELECT m.*, f.*
               FROM tb_member m
               LEFT JOIN tb_memberregistration_feesandcontribution f ON
m.employeeID = f.employeeID
               WHERE m.employeeID = ?

//fetch loan data
SELECT
    SUM(CASE WHEN l.loanType = 'AL-BAI' THEN l.amountRequested ELSE 0
END) as alBai,
```

```
    SUM(CASE WHEN l.loanType = 'AL-INAH' THEN l.amountRequested ELSE 0
END) as alnnah,
    SUM(CASE WHEN l.loanType = 'B/PULIH KENDERAAN' THEN
l.amountRequested ELSE 0 END) as bPulihKenderaan,
    SUM(CASE WHEN l.loanType = 'ROAD TAX & INSURAN' THEN
l.amountRequested ELSE 0 END) as roadTaxInsurance
    FROM tb_loan l
    JOIN tb_loanapplication la ON l.loanApplicationID = la.loanApplicationID
    WHERE l.employeeID = ? AND la.loanStatus = 'Diluluskan'

//view monthly statement
//fetch month
SELECT DISTINCT YEAR(transDate) as year, MONTH(transDate) as month
        FROM tb_transaction
        WHERE employeeID = ?
        ORDER BY year DESC, month DESC

//fetch member detail
SELECT m.*, f.*
               FROM tb_member m
               LEFT JOIN tb_memberregistration_feesandcontribution f ON
m.employeeID = f.employeeID
               WHERE m.employeeID = ?

//fetch first time payment
SELECT MIN(transDate) as first_payment
                     FROM tb_transaction
                     WHERE employeeID = ?

//fetch transaction detail
SELECT t.transDate, t.transType, t.transAmt
        FROM tb_transaction t
        WHERE t.employeeID = ?
        AND MONTH(t.transDate) = ?
        AND YEAR(t.transDate) = ?
if (!$is_first_month) {
    $sql .= " AND t.transType NOT IN ('Entry Fee', 'Deposit')";
}

$sql .= " ORDER BY t.transDate DESC";


//fetch loan detail
SELECT
```

```
    l.loanType,
    la.amountRequested,
    la.monthlyInstallments
FROM tb_loan l
JOIN tb_loanapplication la ON l.employeeID = la.employeeID
WHERE l.employeeID = ?
AND la.loanStatus = 'Diluluskan'

//fetch thet transaction for selected month
SELECT SUM(transAmt) as total_paid
                FROM tb_transaction
                WHERE employeeID = ?
                AND transType = 'Bayaran Ba'
                AND (
                   YEAR(transDate) < ?
                   OR (YEAR(transDate) = ? AND MONTH(transDate) <= ?)
                )

//fetch the amount for selected month
SELECT
    SUM(CASE WHEN transType = 'Simpanan-M' THEN transAmt ELSE 0 END)
as modalShare,
    SUM(CASE WHEN transType = 'Simpanan-Y' THEN transAmt ELSE 0 END)
as feeCapital,
    SUM(CASE WHEN transType = 'Simpanan-T' THEN transAmt ELSE 0 END)
as contribution,
    SUM(CASE WHEN transType = 'Simpanan-S' THEN transAmt ELSE 0 END)
as fixedDeposit
FROM tb_transaction
WHERE employeeID = ?
AND (
    YEAR(transDate) < ?
    OR (YEAR(transDate) = ? AND MONTH(transDate) <= ?)
)
AND transType IN ('Simpanan-M', 'Simpanan-Y', 'Simpanan-T', 'Simpanan-S')

//track the transaction history
//fetch the first time payment
SELECT MIN(transDate) as first_payment
                 FROM tb_transaction
                 WHERE employeeID = ?

//fetch the transaction detail
SELECT t.transDate, t.transType, t.transAmt
```

```
        FROM tb_transaction t
        WHERE t.employeeID = ?
        AND MONTH(t.transDate) = ?
        AND YEAR(t.transDate) = ?
if (!$is_first_month) {
    $sql .= " AND t.transType NOT IN ('Entry Fee', 'Deposit')";
}

$sql .= " ORDER BY t.transDate DESC";


//fetch the date of first time payment
SELECT MIN(transDate) as first_payment
                     FROM tb_transaction
                     WHERE employeeID = ?
```

5.0 Summary

The Koperasi KADA Online System, a revolutionary project that aims to modernise the cooperative's operations by substituting the current manual processes with an automated, secure, and user-friendly digital platform, is described in this report along with its design and implementation. The suggested system significantly improves operational efficiency and user satisfaction while addressing the shortcomings of the current system, including its excessive reliance on human resources, error-proneness, and insufficient data security.

The report provides with an overview of Koperasi Kakitangan KADA Kelantan Berhad, emphasising the organization's goal of promoting its members' social and economic well-being. It then gives a thorough rundown of the difficulties with the manual system, such as laborious procedures, a lack of services, and dangers related to physical data storage. Online registration, real-time application tracking, automated loan processing, and secure financial record management are among the solutions offered by the proposed Koperasi KADA Online System.

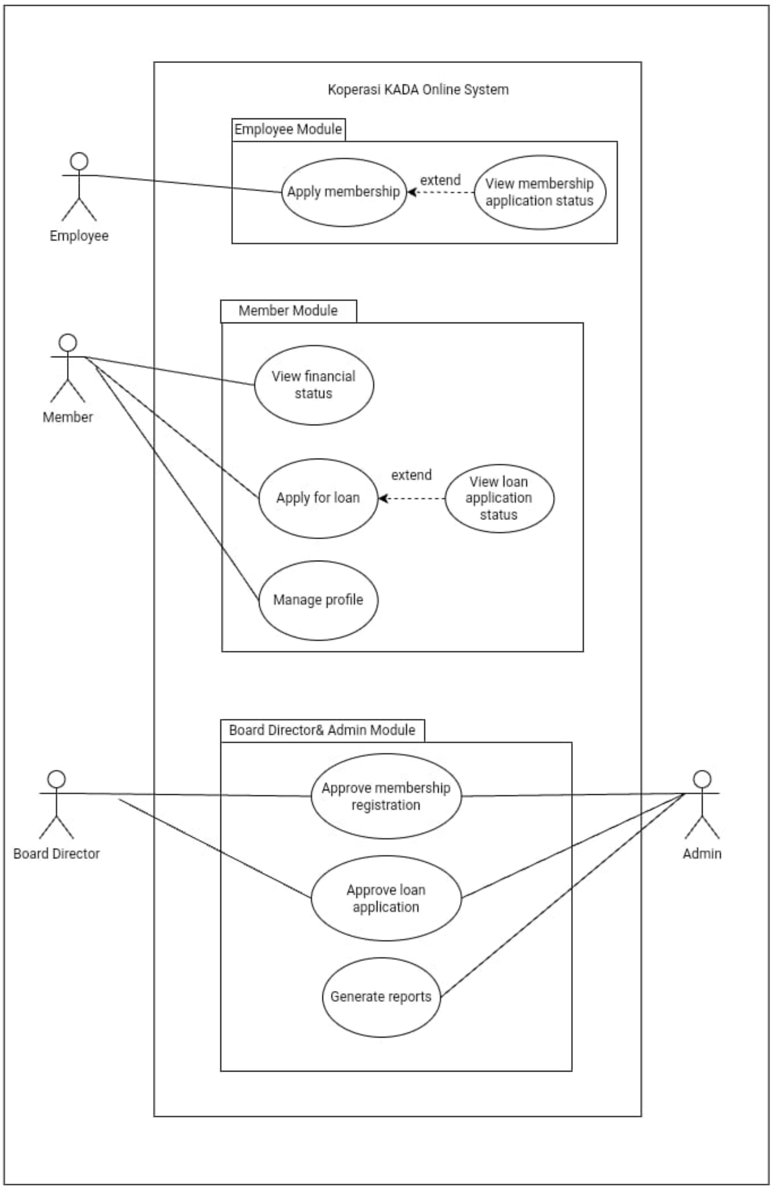
The database design section includes a Global Conceptual Data Model , an Entity-Relationship Diagram (ERD), and a data dictionary to ensure data consistency and integration. The Logical Data Model enhances this design by normalising data structures up to the third normal form (3NF), which improves database performance and reduces redundancy. The Interface Design and SQL Implementation sections demonstrate a user-centric system with tailored interfaces for members and administrators, as well as robust SQL scripts for efficient and secure data management.

By automating key workflows and implementing advanced security measures such as Two-Factor Authentication (2FA), the system reduces human error, workload, and protects sensitive member data. The platform improves the overall user experience by giving members easy access to services and real-time updates, while staff can focus on strategic tasks.

In conclusion, the Koperasi KADA Online System is a significant step forward in the cooperative's digital transformation. This scalable and secure platform aligns with KADA's mission, allowing the organisation to improve services, increase member satisfaction, and achieve operational excellence.

6.0 Appendices

6.1 Use Case Diagram for the system



6.2 Activity Diagrams for every use case identified

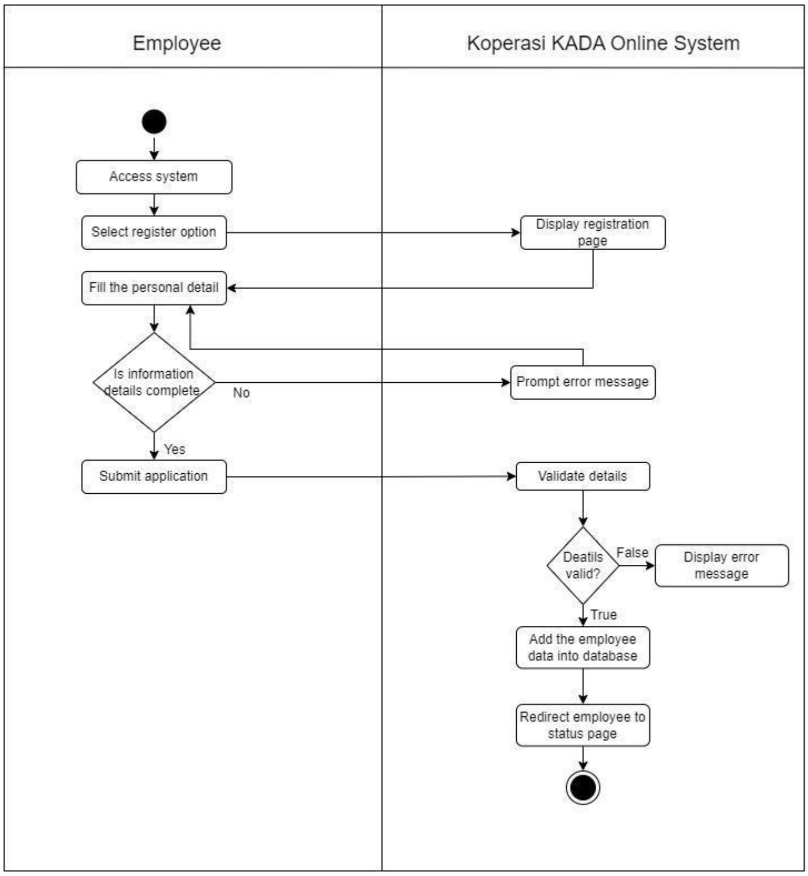


Figure 6.2.1 Activity Diagram for Apply Membership

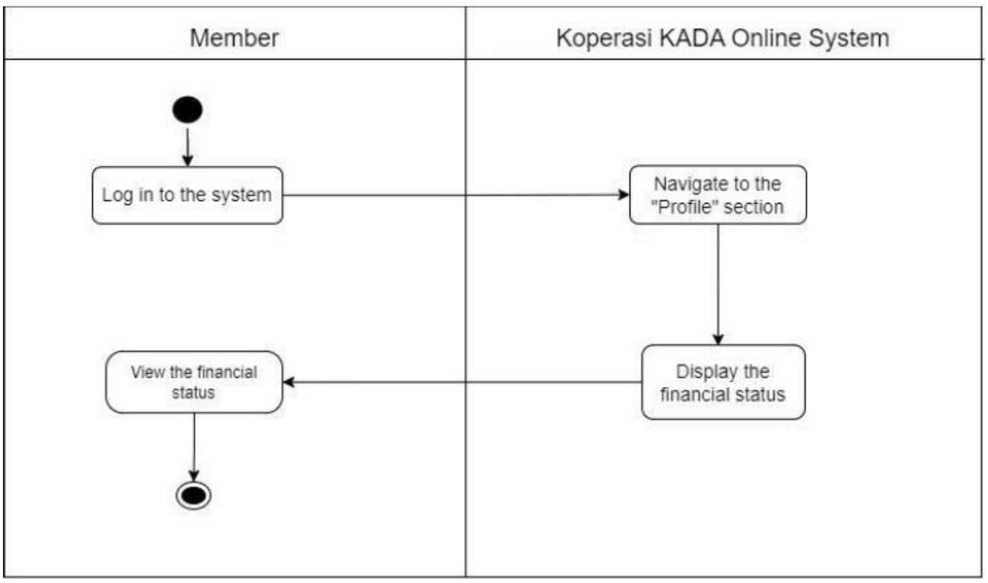


Figure 6.2.2 Activity Diagram for View Financial Status

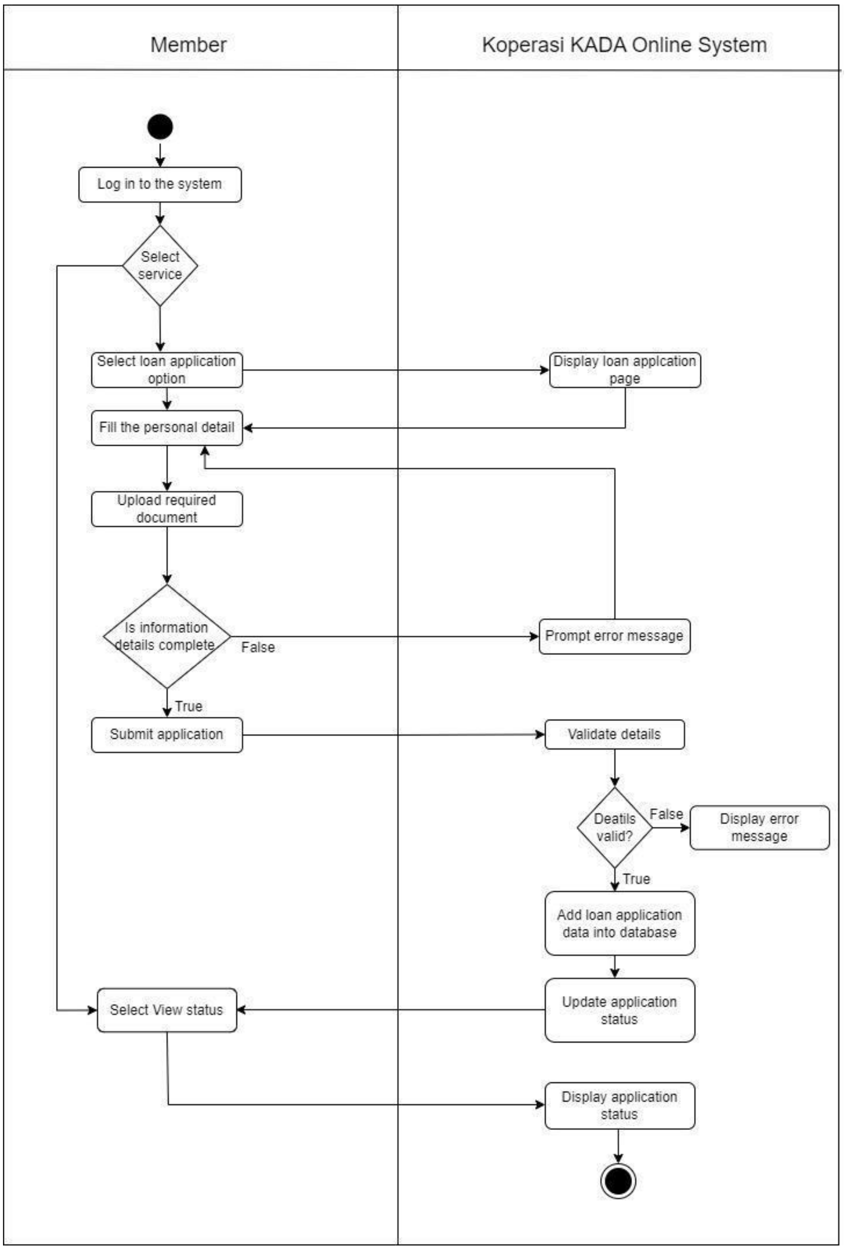


Figure 6.2.3 Activity Diagram for Apply for Loan

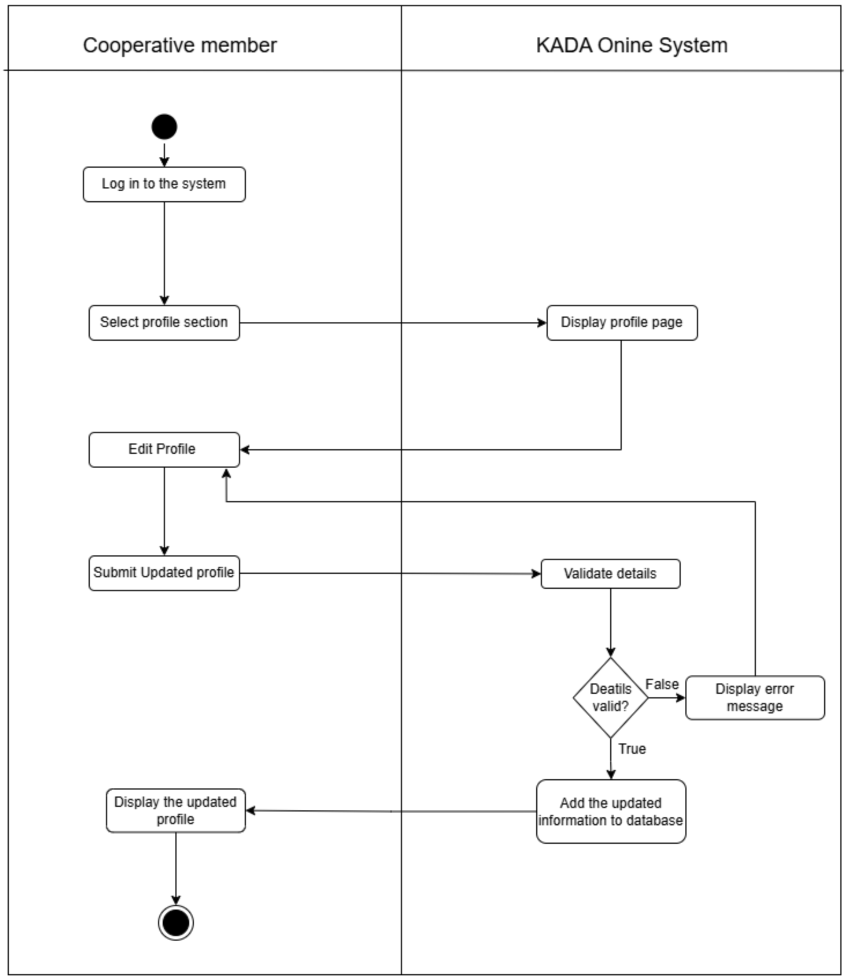


Figure 6.2.4 Activity Diagram for Manage Profile

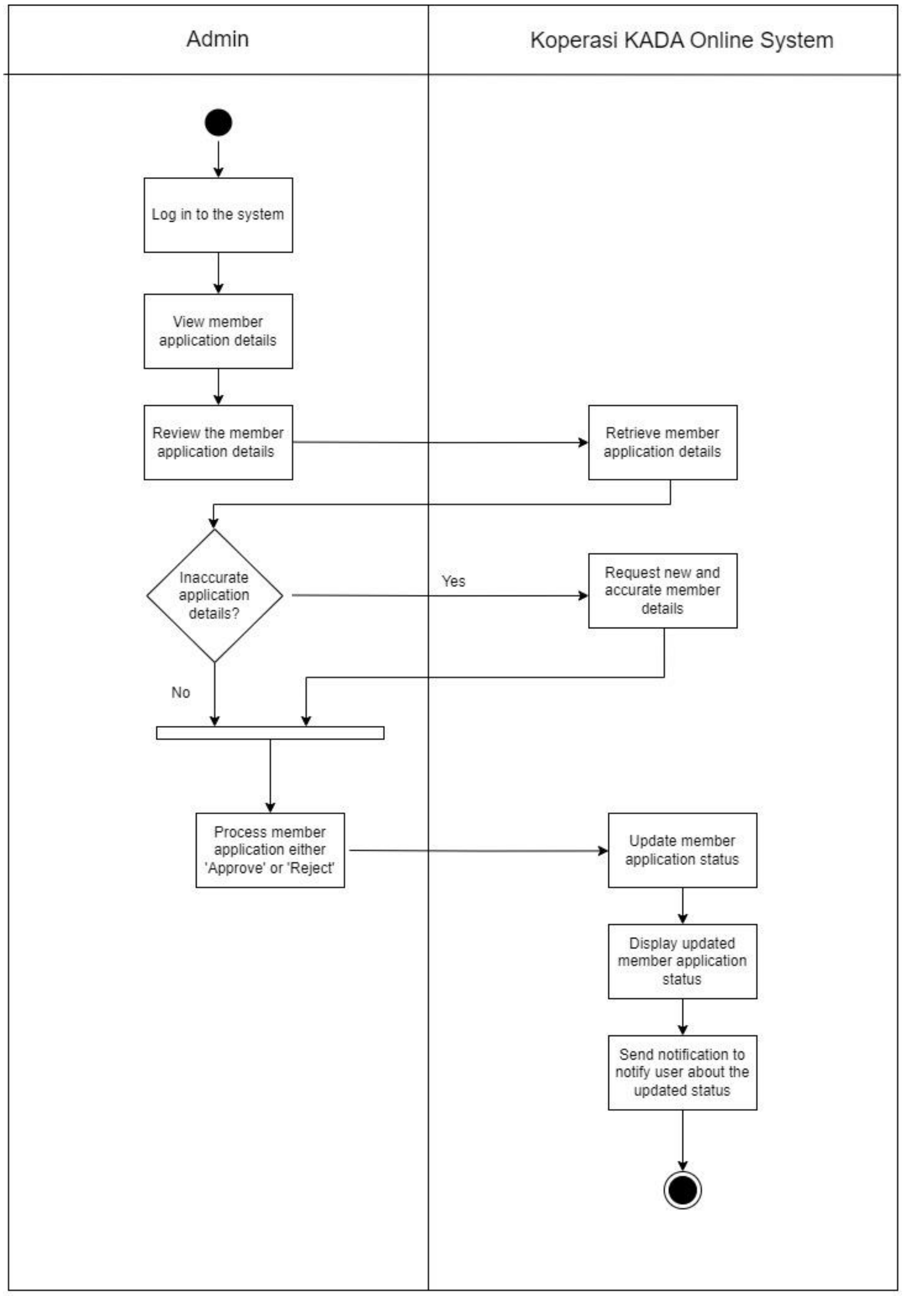


Figure 6.2.5 Activity Diagram for Approve Membership Application

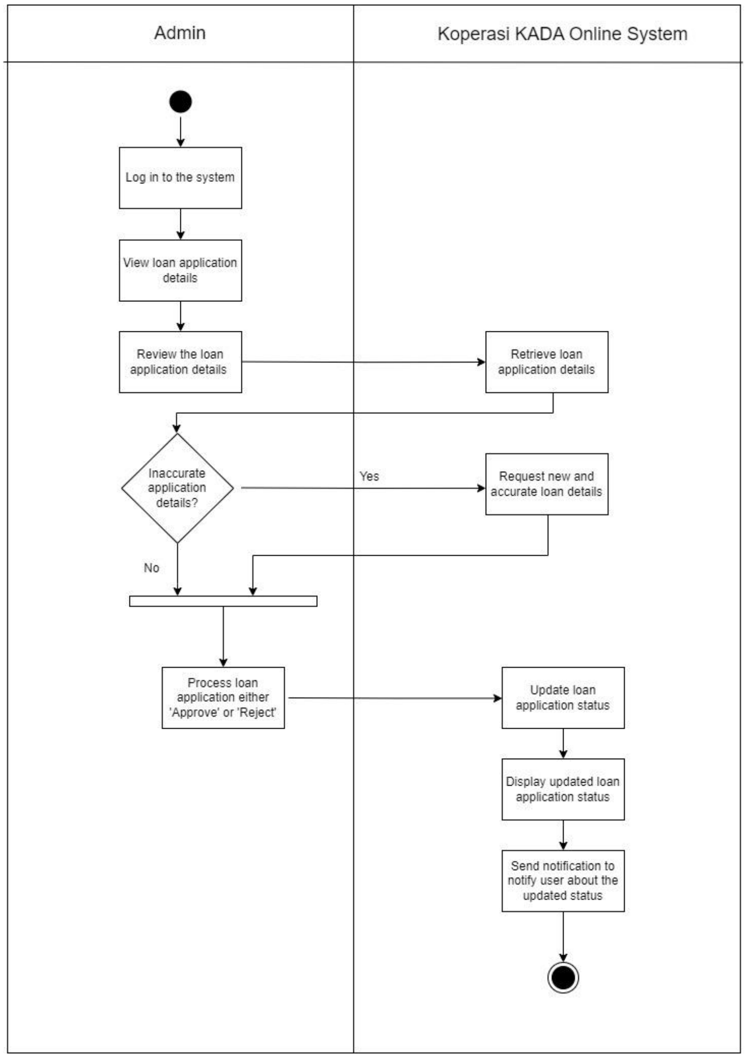


Figure 6.2.6 Activity Diagram for Approve Loan Application

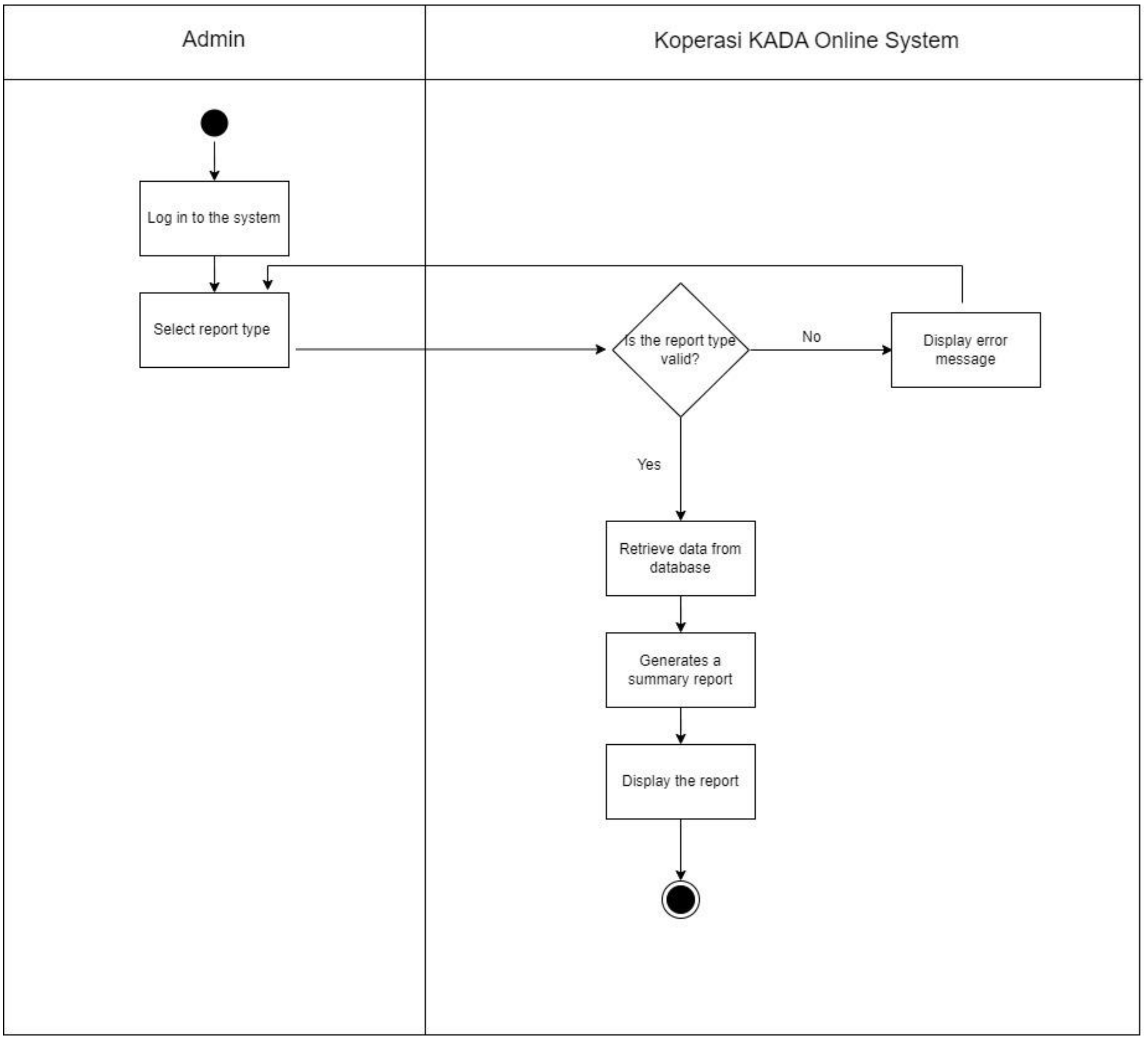


Figure 6.2.7 Activity Diagram for Generate Report

Meeting Log

Meeting #1 (20/1/2025, 8.00pm - 11.00pm)

Our first meeting was on January 20, 2025, at 8 p.m. sharp. The purpose of this meeting is to discuss the report's requirements, which include system introduction, database requirements, global erd, and system implementation. Everyone in the group arrived on time for the meeting, and we discussed the project together.

Meeting #2 (25/1/2025, 9.00pm - 11.00pm)

We held our second meeting on January 25, 2025, at 9:00 p.m. The purpose of this meeting is to conduct a final check on the report that we completed before submitting it. Every group member took part in this discussion, and we checked to confirm if we had completed the report appropriately.